Lab 3: Reinforcement Learning

Your Name

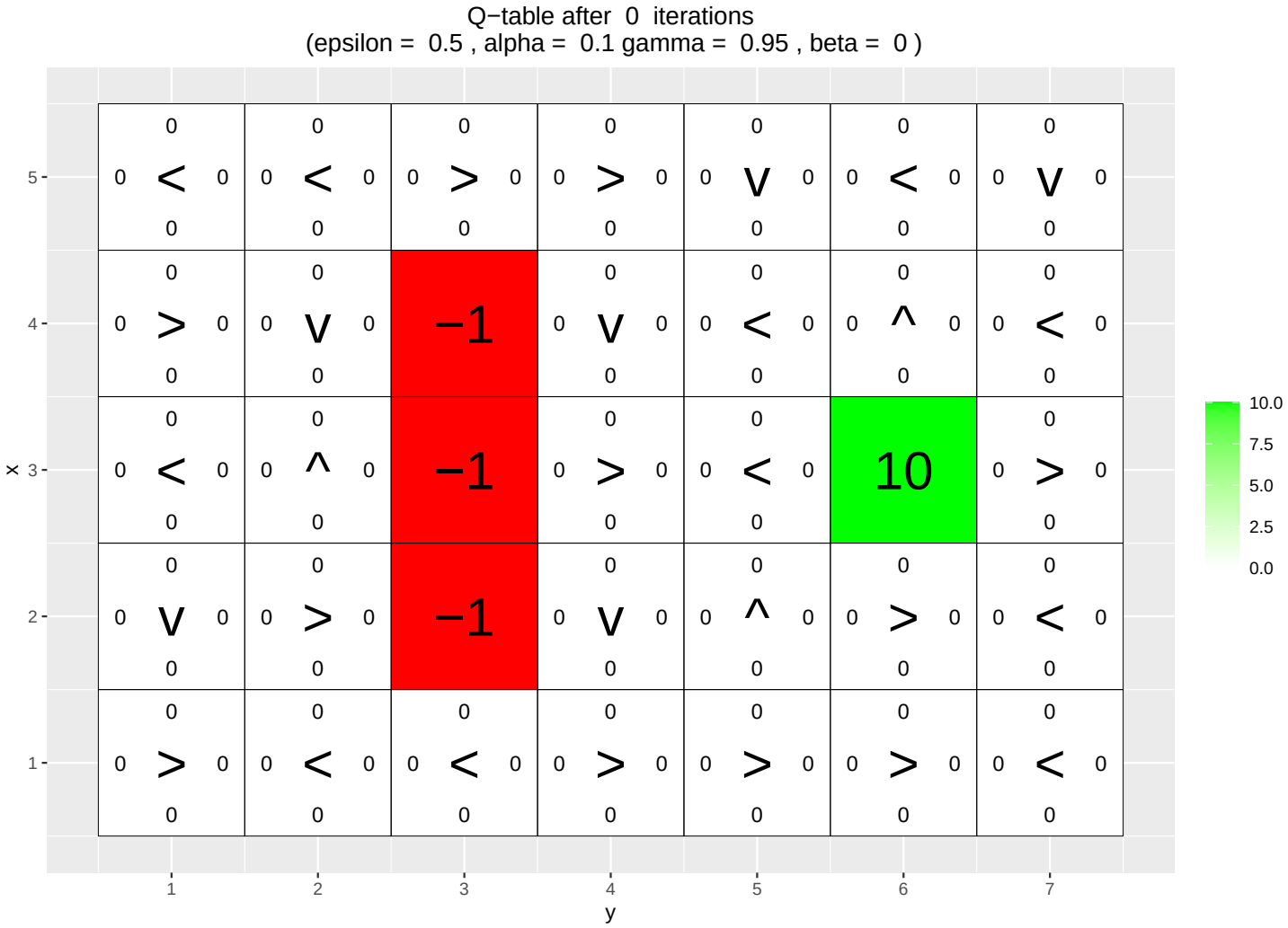
2026-10-05

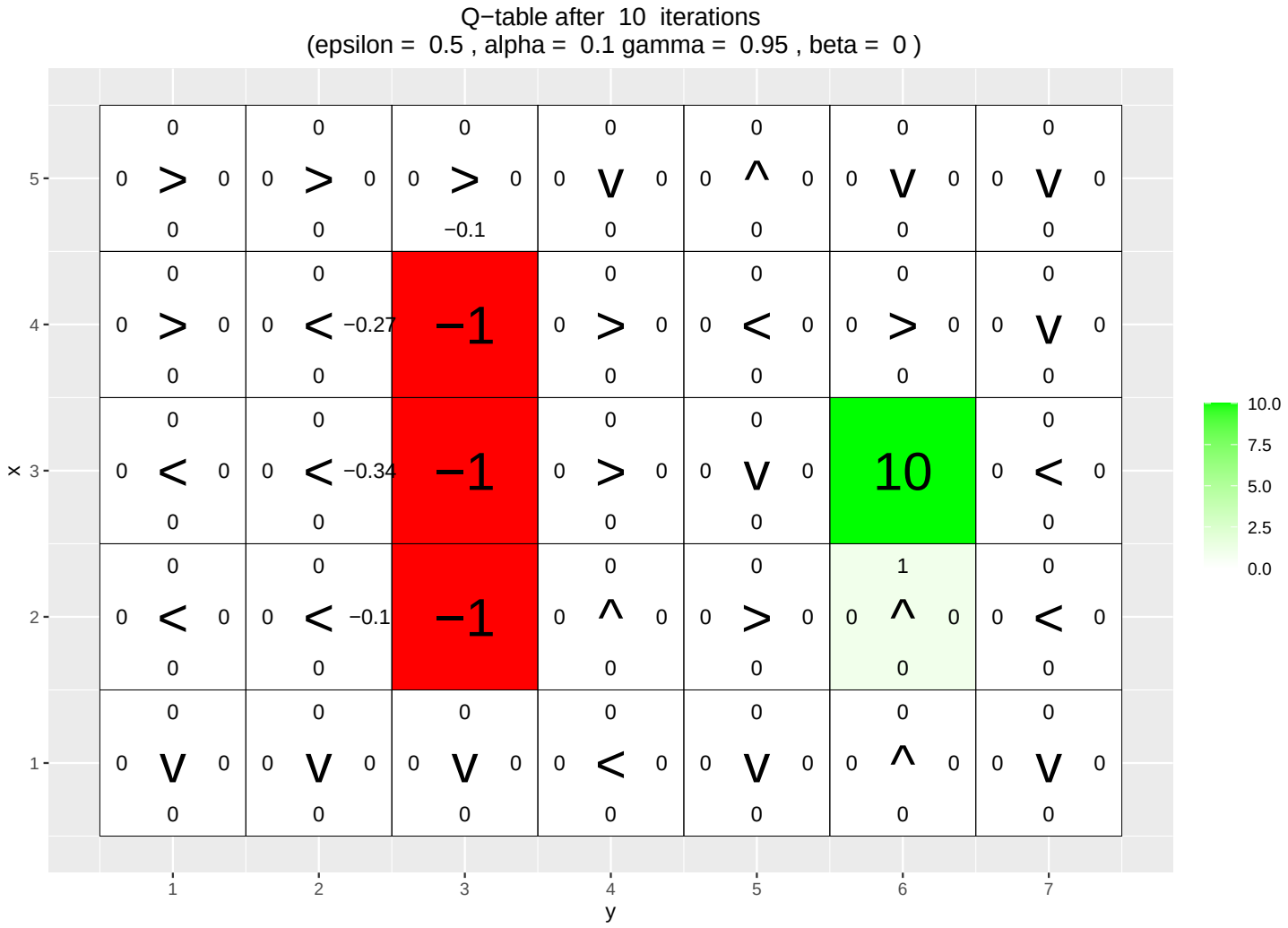
1. Q-learning

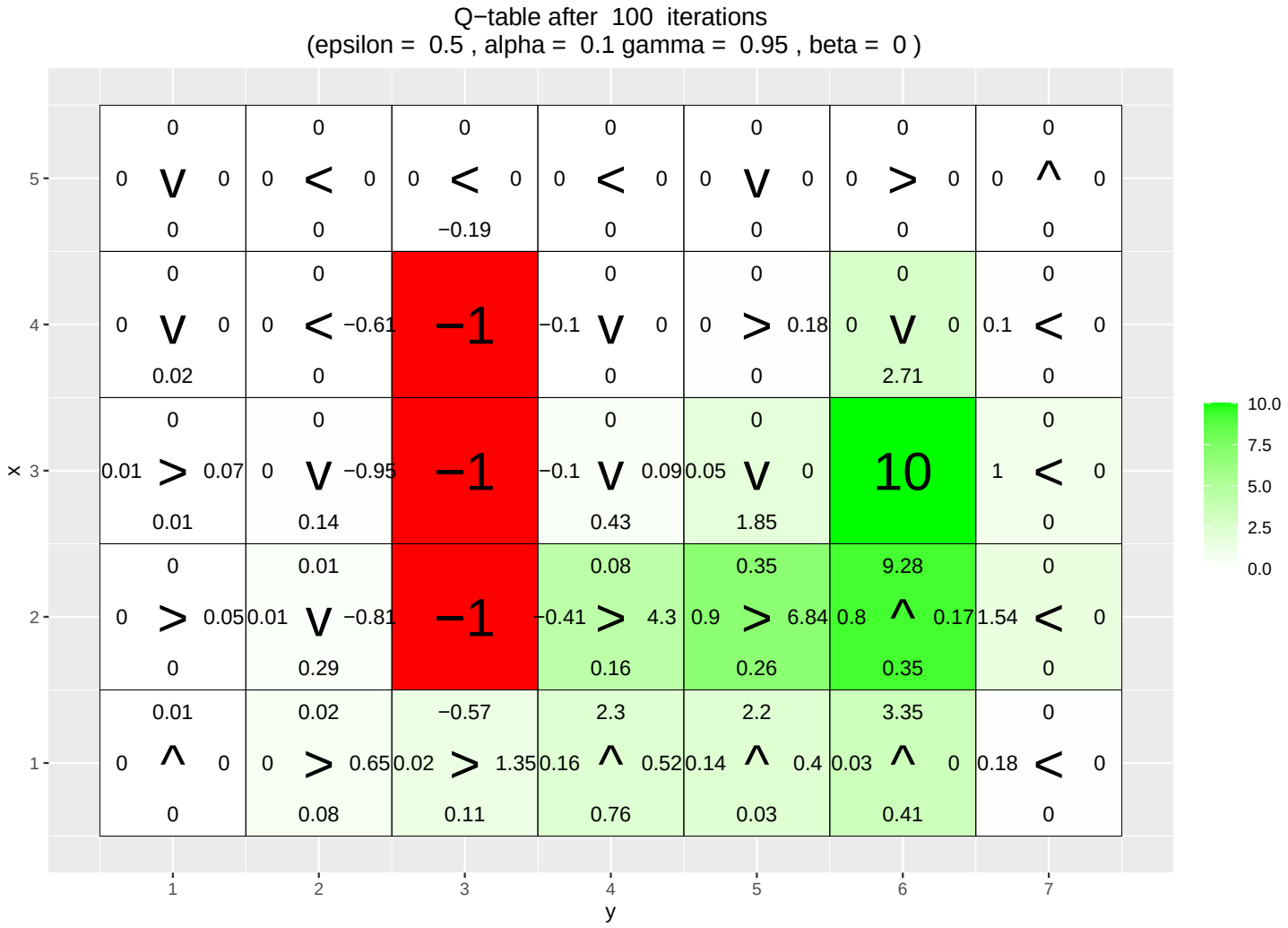
The grid-world is a fully observable, episodic Markov decision process. The agent receives the reward of the state it enters, and any non-zero-reward state ends the episode. The Q-learning implementation, including the epsilon-greedy policy, random tie-breaking, transition model and temporal-difference update, is listed in Appendix A.1.

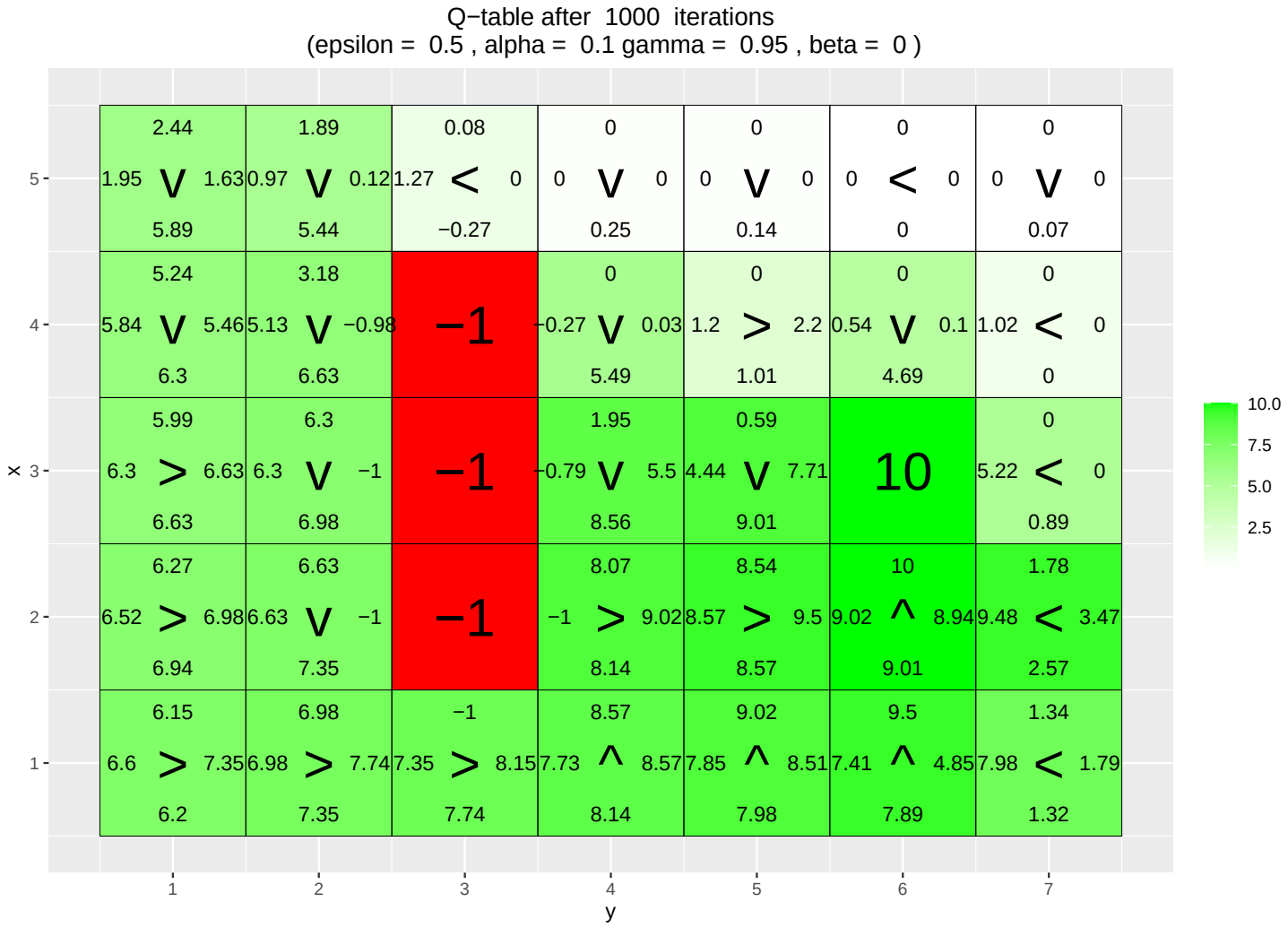
1.1 Environment A

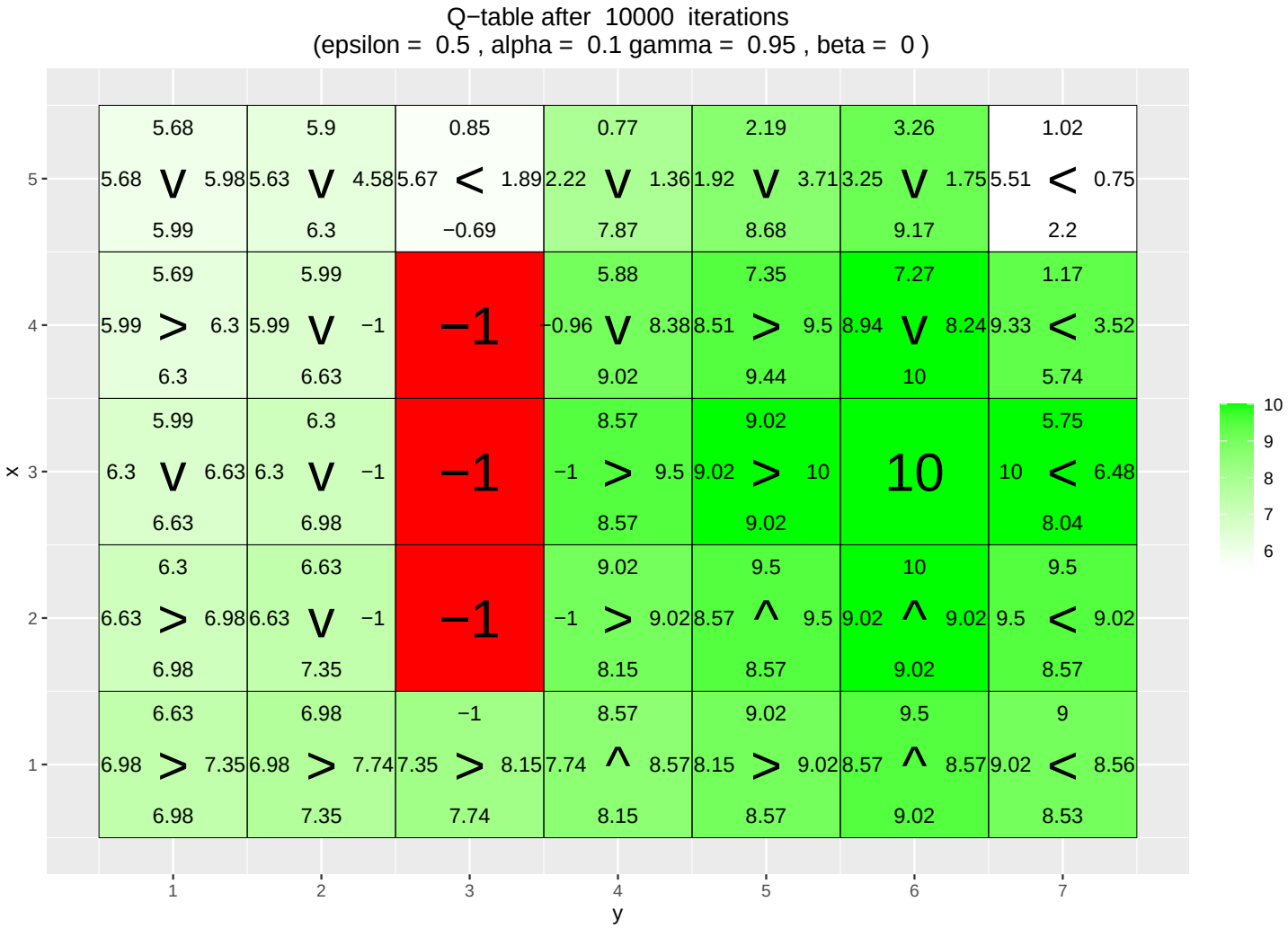
The agent starts each episode at state (3, 1). The goal reward is +10 at (3, 6), and states (2, 3), (3, 3) and (4, 3) have reward -1. The Q-table and learned greedy policy after training are shown below.




Q−table after 10 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.95 , beta = 0)
x
y
1
2
3
4
5
6
7
10.0
7.5
5.0
2.5
0.0
−0.1
−0.27
−0.34
−0.1
−1
10


Q−table after 100 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.95 , beta = 0)
x
y
5
4
3
2
1
1
2
3
4
5
6
7
10.0
7.5
5.0
2.5
0.0
0 V 0 0
0 < 0 0
0 < 0 −0.19
0 < 0 0
0 V 0 0
0 > 0 0
0 ^ 0 0
0 V 0 0.02
0 < 0 −0.61 0
−1
−0.1 V 0 0
0 > 0.18 0
0 V 0 2.71
0.1 < 0 0
0.01 > 0.07 0.01
0 V −0.95 0.14
−1
−0.1 V 0.09 0.43
0.05 V 0 1.85
10
1 < 0 0
0 > 0.05 0
0.01 V −0.81 0.29
−1
−0.41 > 4.3 0.16
0.9 > 6.84 0.26
0.8 ^ 0.17 0.35
1.54 < 0 0
0.01 ^ 0 0
0 > 0.65 0.08
−0.57 0.02 > 1.35 0.11
2.3 0.16 ^ 0.52 0.76
2.2 0.14 ^ 0.4 0.03
3.35 0.03 ^ 0 0.41
0 0.18 < 0 0


Q−table after 1000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.95 , beta = 0)
2.44
1.95 V 1.63
5.89
1.89
0.97 V 0.12
5.44
0.08
1.27 < 0
−0.27
0
0 V 0
0.25
0
0 V 0
0.14
0
0 < 0
0
0
0 V 0
0.07
5.24
5.84 V 5.46
6.3
3.18
5.13 V −0.98
6.63
−1
0
−0.27 V 0.03
5.49
0
1.2 > 2.2
1.01
0
0.54 V 0.1
4.69
0
1.02 < 0
0
5.99
6.3 > 6.63
6.63
6.3
6.3 V −1
6.98
−1
1.95
−0.79 V 5.5
8.56
0.59
4.44 V 7.71
9.01
10
0
5.22 < 0
0.89
6.27
6.52 > 6.98
6.94
6.63
6.63 V −1
7.35
−1
8.07
−1 > 9.02
8.14
8.54
8.57 > 9.5
8.57
10
9.02 ^ 8.94
9.01
1.78
9.48 < 3.47
2.57
6.15
6.6 > 7.35
6.2
6.98
6.98 > 7.74
7.35
−1
7.35 > 8.15
7.74
8.57
7.73 ^ 8.57
8.14
9.02
7.85 ^ 8.51
7.98
9.5
7.41 ^ 4.85
7.89
1.34
7.98 < 1.79
1.32
x
y
1
2
3
4
5
6
7
10.0
7.5
5.0
2.5



What has the agent learned after the first 10 episodes?

After only 10 episodes, the agent has already learned to avoid the -1 on the left side near the starting state. But the majority of states aare zeros, meaning the agent has not encountered them yet.

Is the final greedy policy optimal for all states? Why or why not?

Not necessarily. Training always starts at (3, 1), and 10,000 episodes do not guarantee that every state-action pair is explored enough to learn its optimal value. The policy is optimal on commonly visited routes but not optimal from every state, top left corner for example.

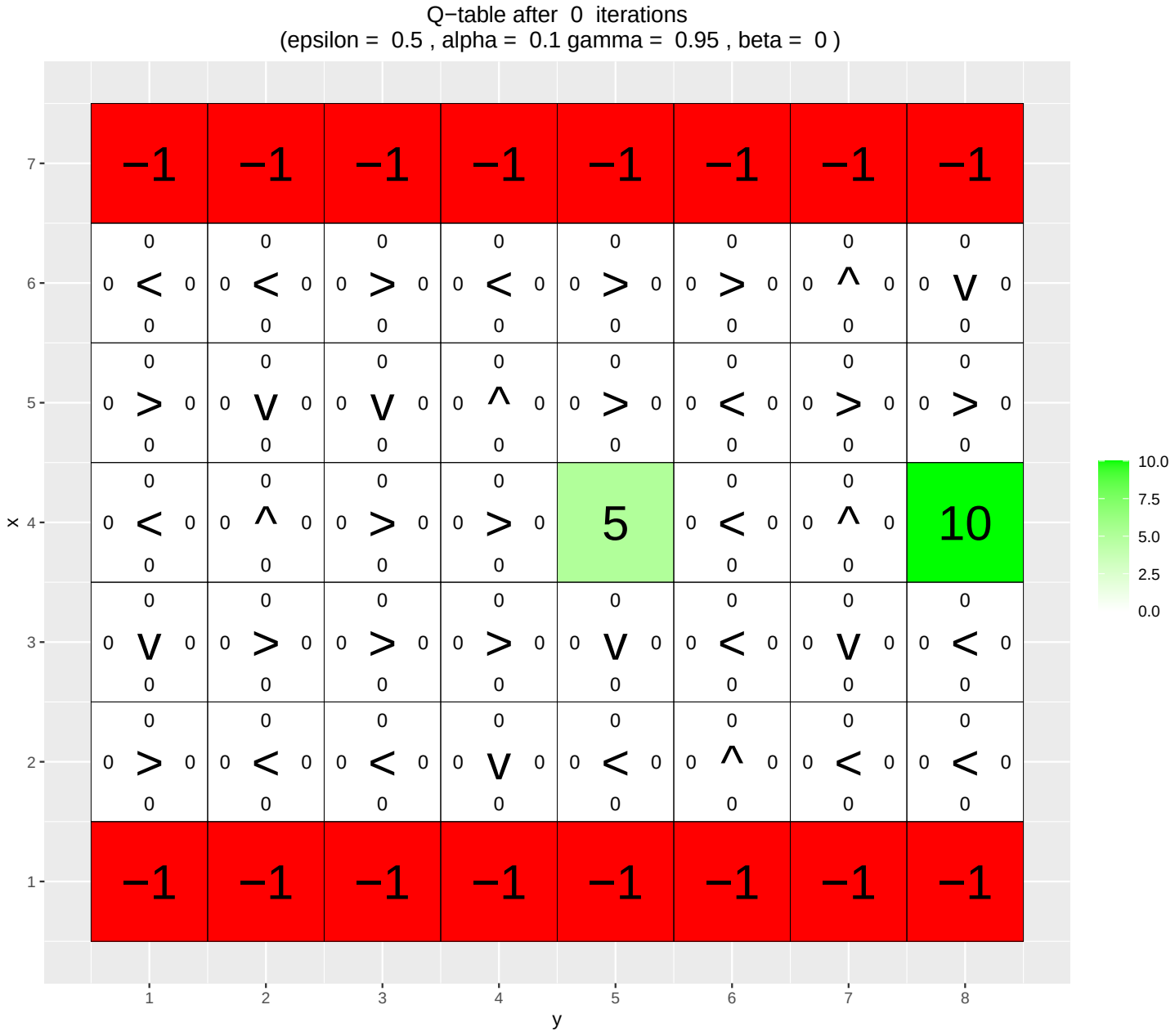
Do the learned Q-values reflect the multiple paths to the positive reward? If not, what could be done?

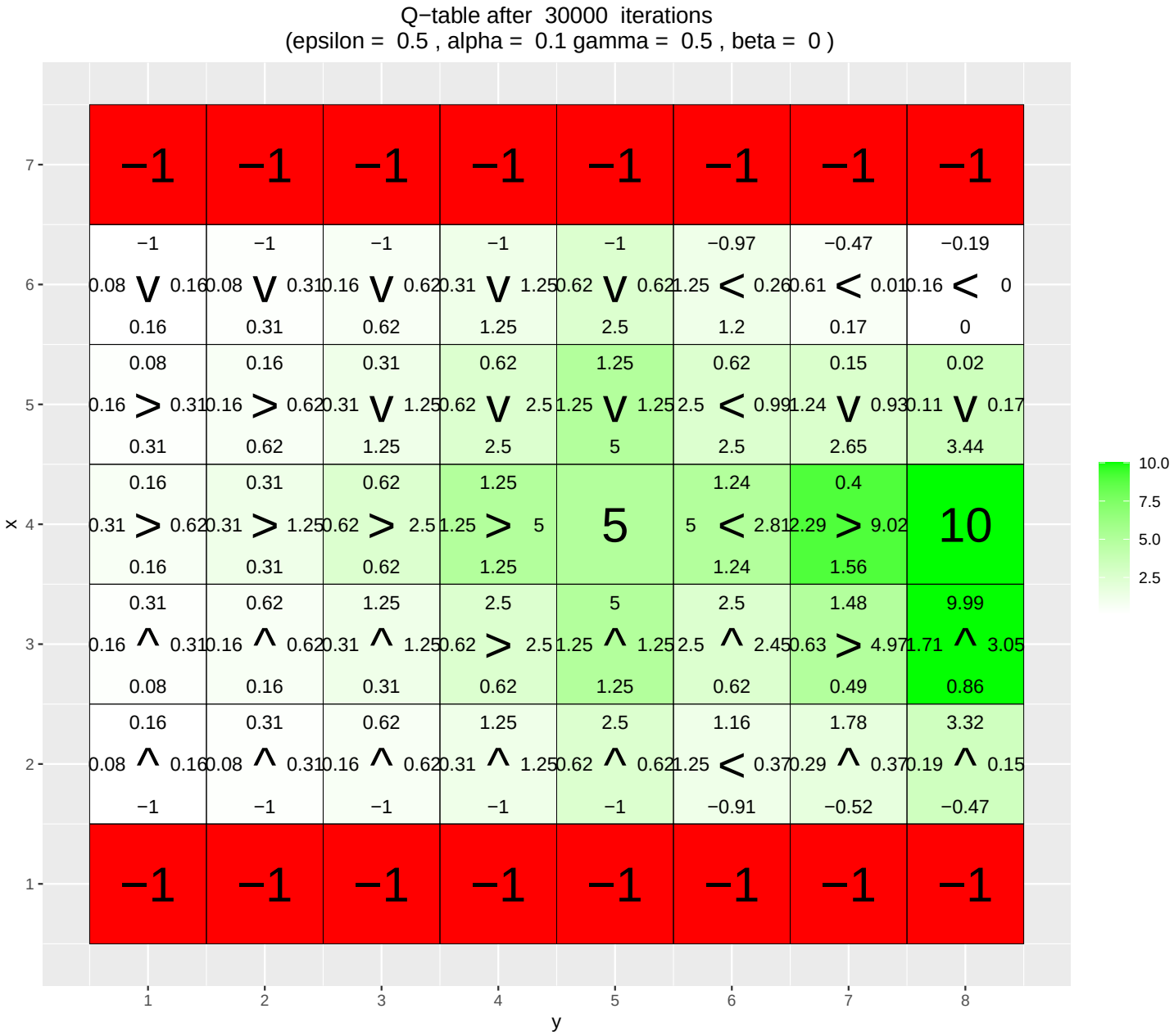
The two routes around the negative-reward states can be learned only if both receive enough exploration. With a fixed start state the agent clearly prefer the lower route, leaving the alternative not as well estimated. More training, exploration, or changing the starting state would give both routes more coverage.

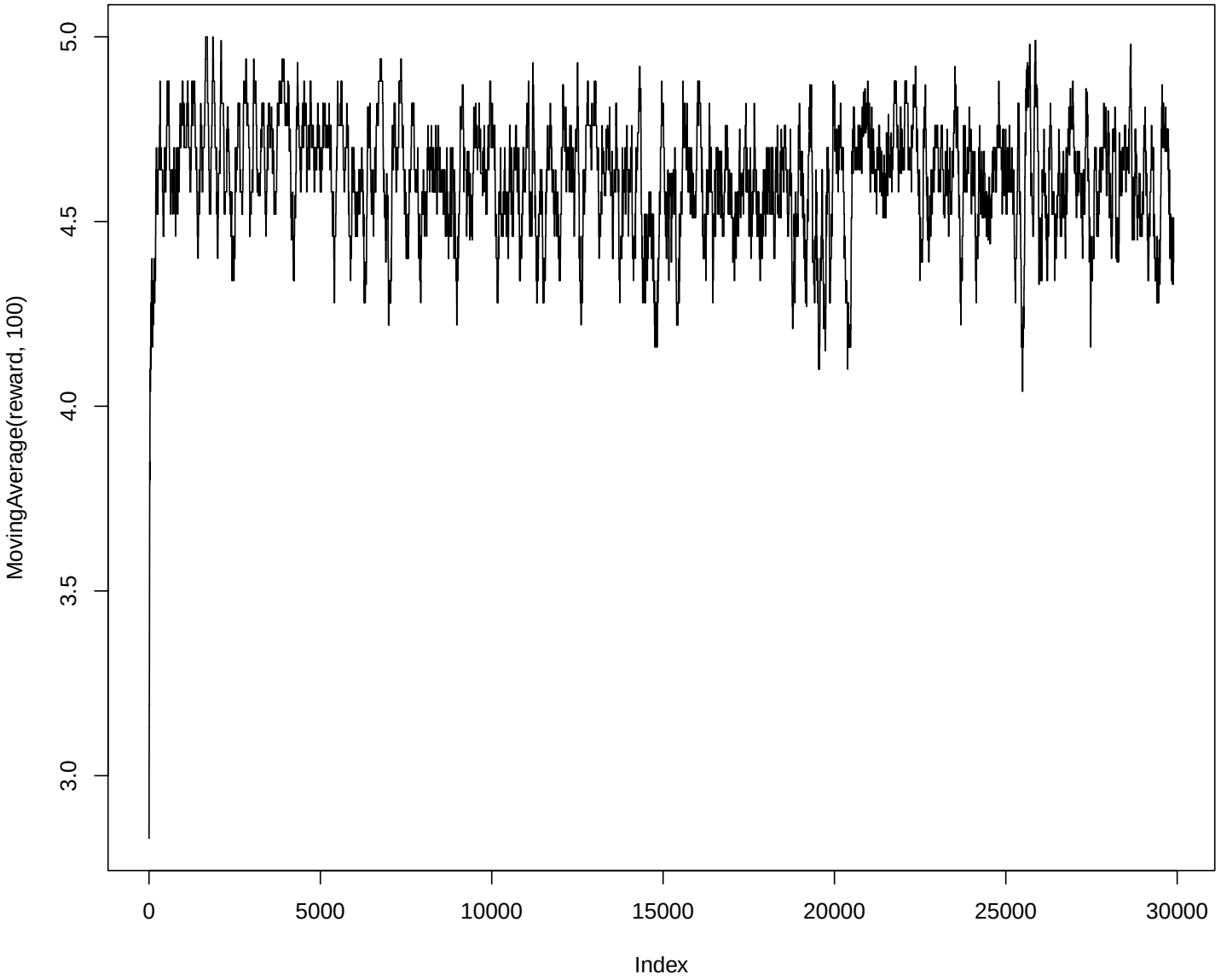
1.2 Environment B: effects of epsilon and gamma

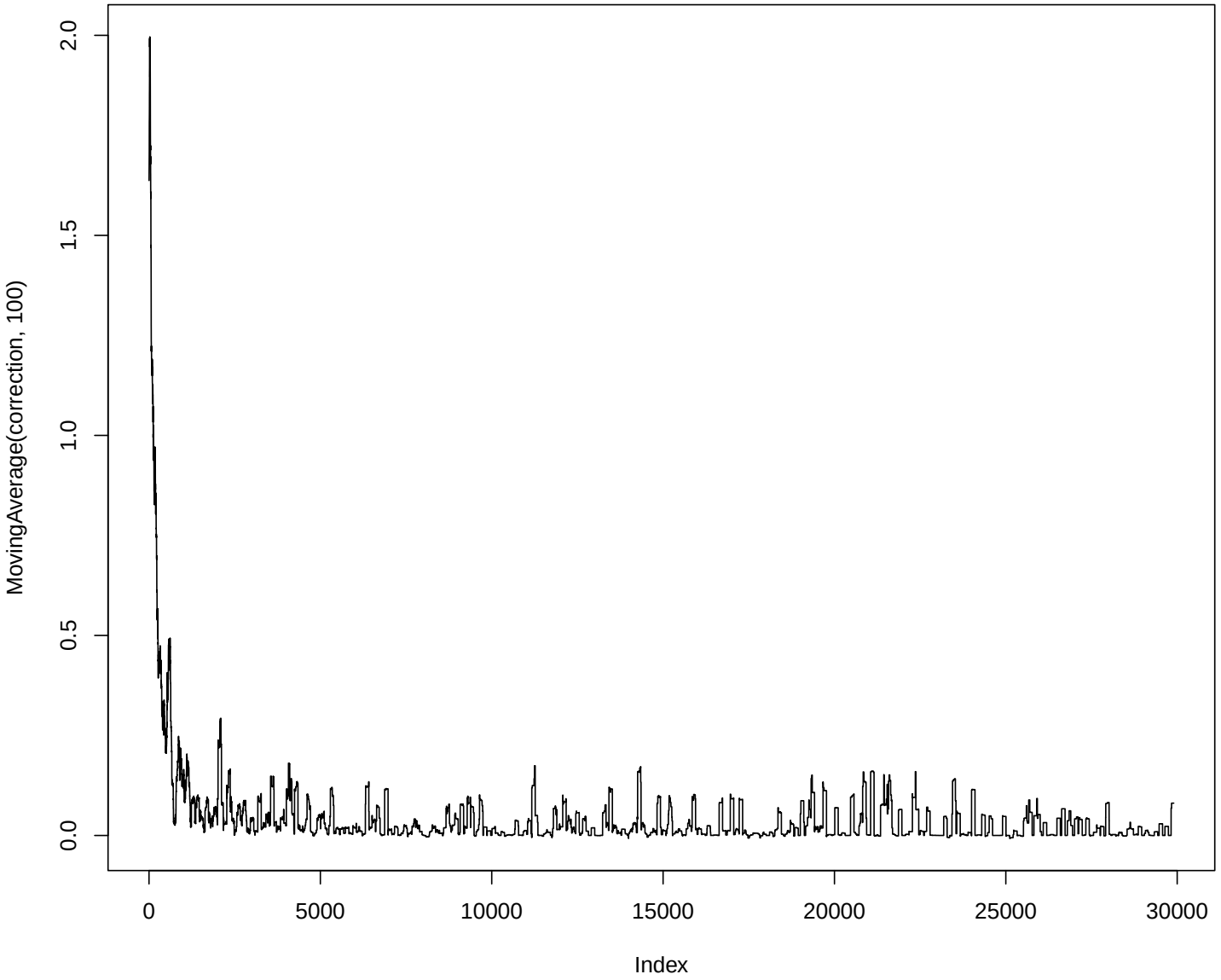
Environment B has negative-reward top and bottom rows, a +5 reward and a +10 reward. The experiment compares gamma values 0.5, 0.75 and 0.95 at epsilon 0.5 and 0.1. Here we look at the effects of epsilon (exploration) and gamma (future reward discount).

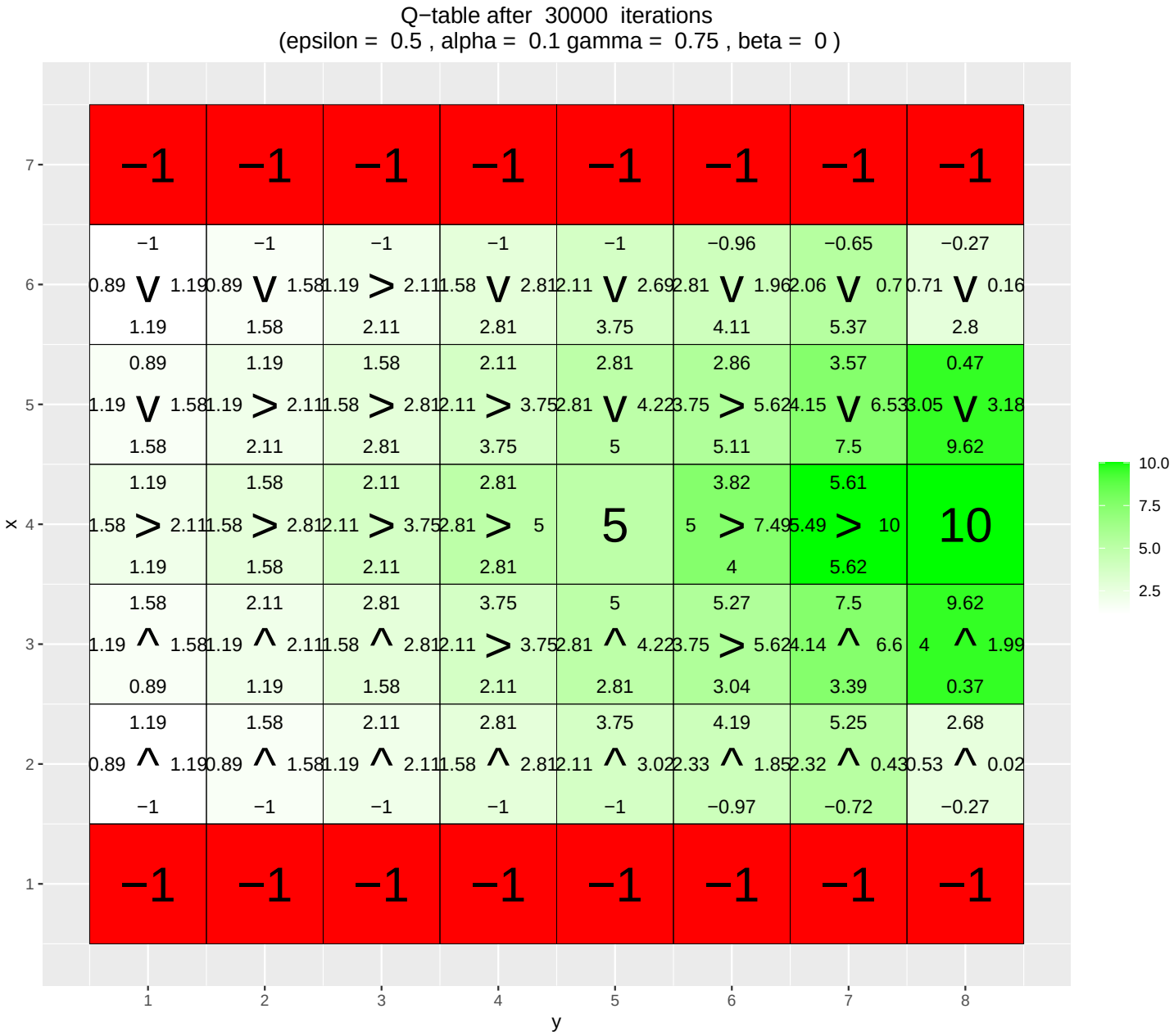
Effects of gamma with epsilon = 0.5

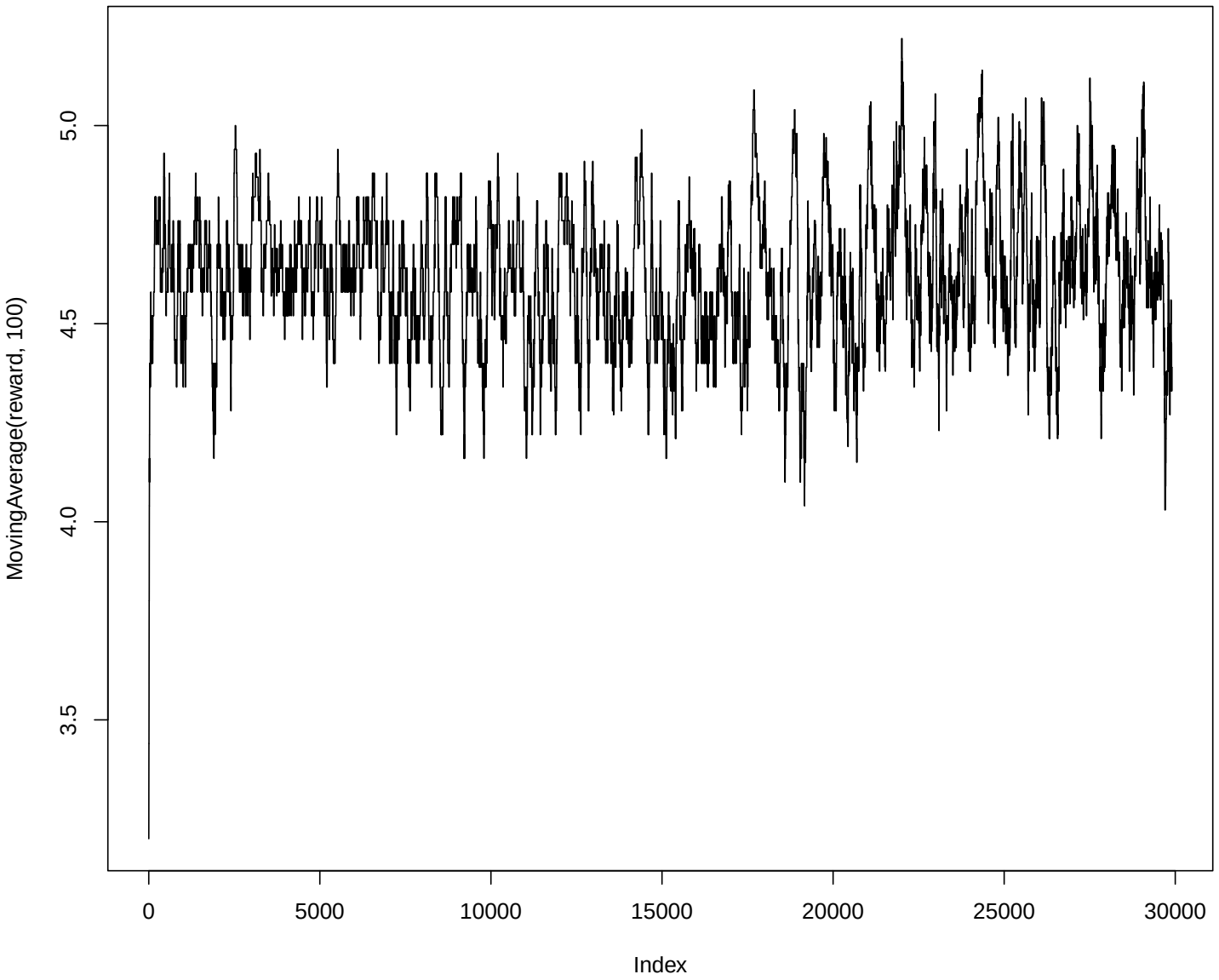


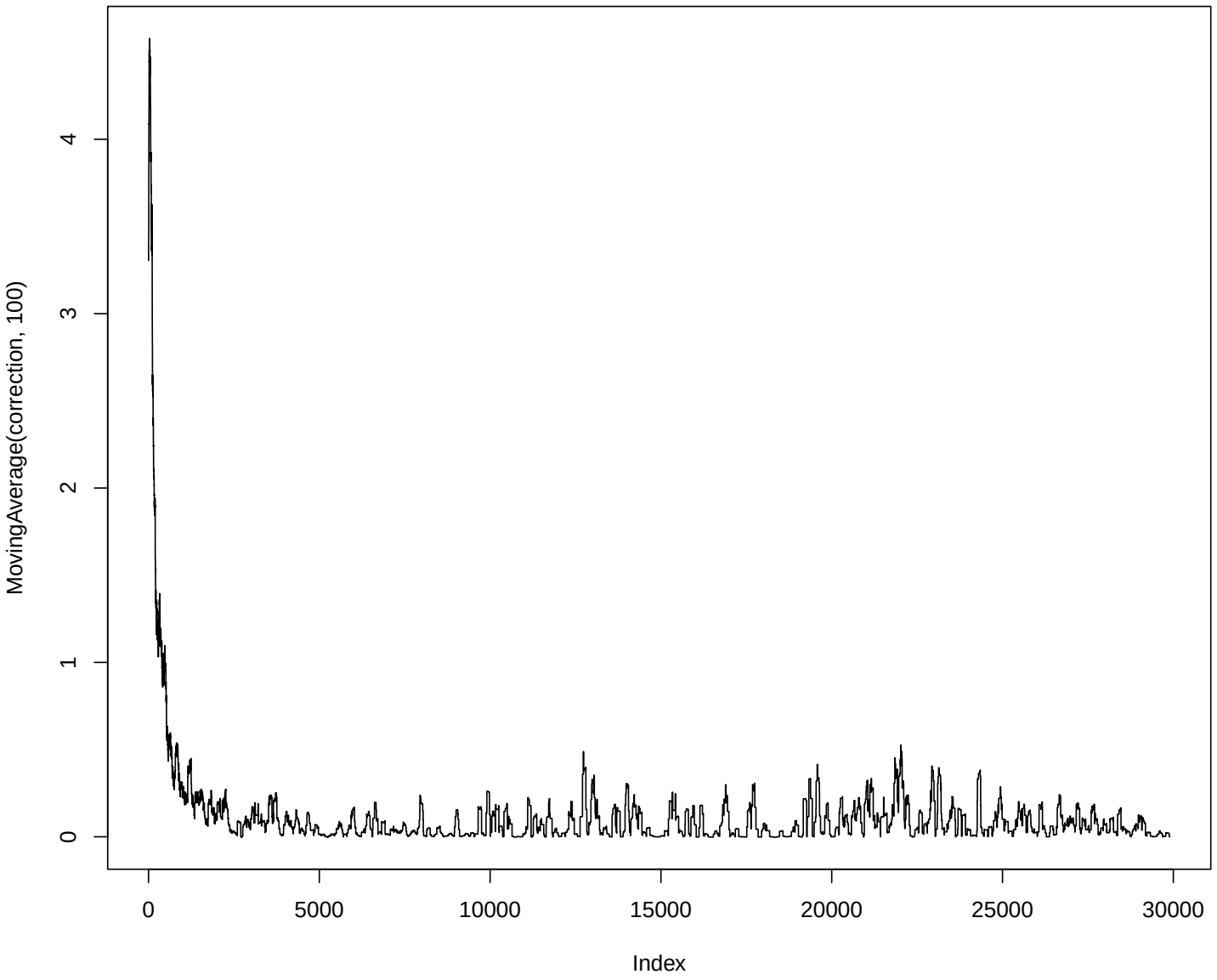

Q−table after 30000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.5 , beta = 0)
−1 −1 −1 −1 −1 −1 −1 −1
−1 −1 −1 −1 −1 −0.97 −0.47 −0.19
0.08 V 0.16 0.08 V 0.31 0.16 V 0.62 0.31 V 1.25 0.62 V 0.62 1.25 < 0.26 0.61 < 0.01 0.16 < 0
0.16 0.31 0.62 1.25 2.5 1.2 0.17 0
0.08 0.16 0.31 0.62 1.25 0.62 0.15 0.02
0.16 > 0.31 0.16 > 0.62 0.31 V 1.25 0.62 V 2.5 1.25 V 1.25 2.5 < 0.99 1.24 V 0.93 0.11 V 0.17
0.31 0.62 1.25 2.5 5 2.5 2.65 3.44
0.16 0.31 0.62 1.25 1.24 0.4
0.31 > 0.62 0.31 > 1.25 0.62 > 2.5 1.25 > 5 5 5 < 2.81 2.29 > 9.02 10
0.16 0.31 0.62 1.25 1.24 1.56
0.31 0.62 1.25 2.5 5 2.5 1.48 9.99
0.16 ^ 0.31 0.16 ^ 0.62 0.31 ^ 1.25 0.62 > 2.5 1.25 ^ 1.25 2.5 ^ 2.45 0.63 > 4.97 1.71 ^ 3.05
0.08 0.16 0.31 0.62 1.25 0.62 0.49 0.86
0.16 0.31 0.62 1.25 2.5 1.16 1.78 3.32
0.08 ^ 0.16 0.08 ^ 0.31 0.16 ^ 0.62 0.31 ^ 1.25 0.62 ^ 0.62 1.25 < 0.37 0.29 ^ 0.37 0.19 ^ 0.15
−1 −1 −1 −1 −1 −0.91 −0.52 −0.47
−1 −1 −1 −1 −1 −1 −1 −1
x
y
10.0
7.5
5.0
2.5

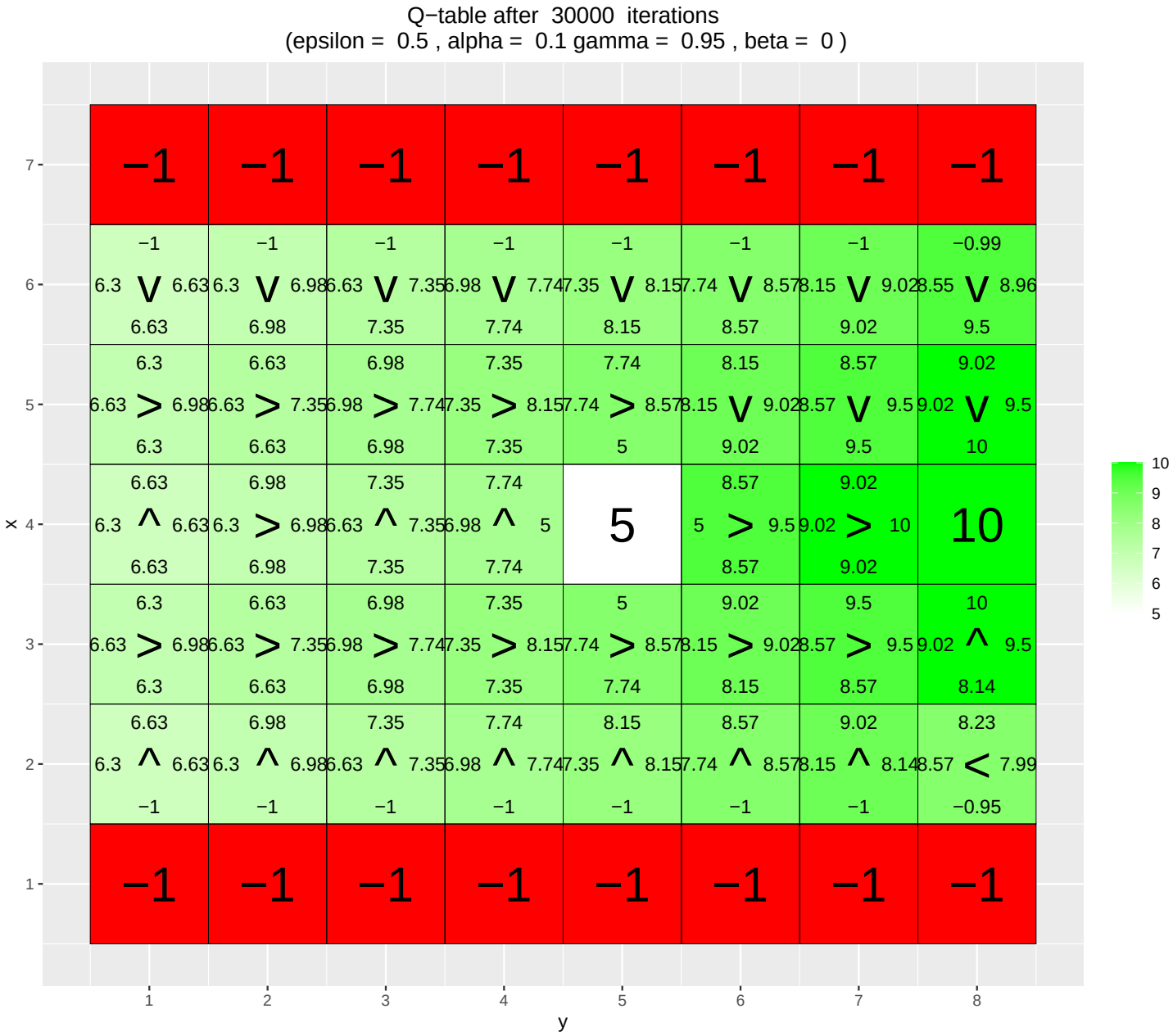

MovingAverage(reward, 100)
5.0
4.5
4.0
3.5
3.0
0
5000
10000
15000
20000
25000
30000
Index

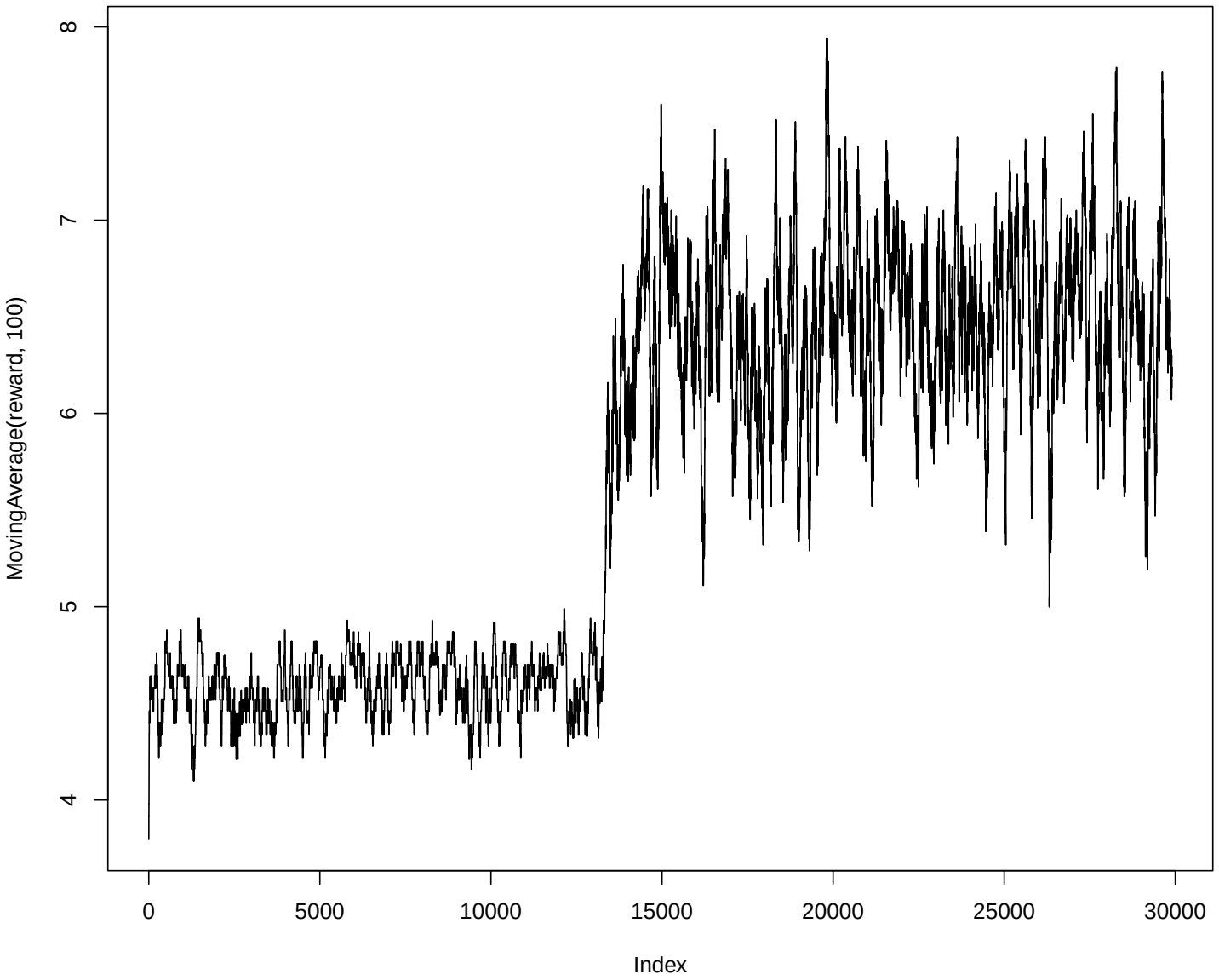

MovingAverage(correction, 100)
0.0
0.5
1.0
1.5
2.0
0
5000
10000
15000
20000
25000
30000
Index

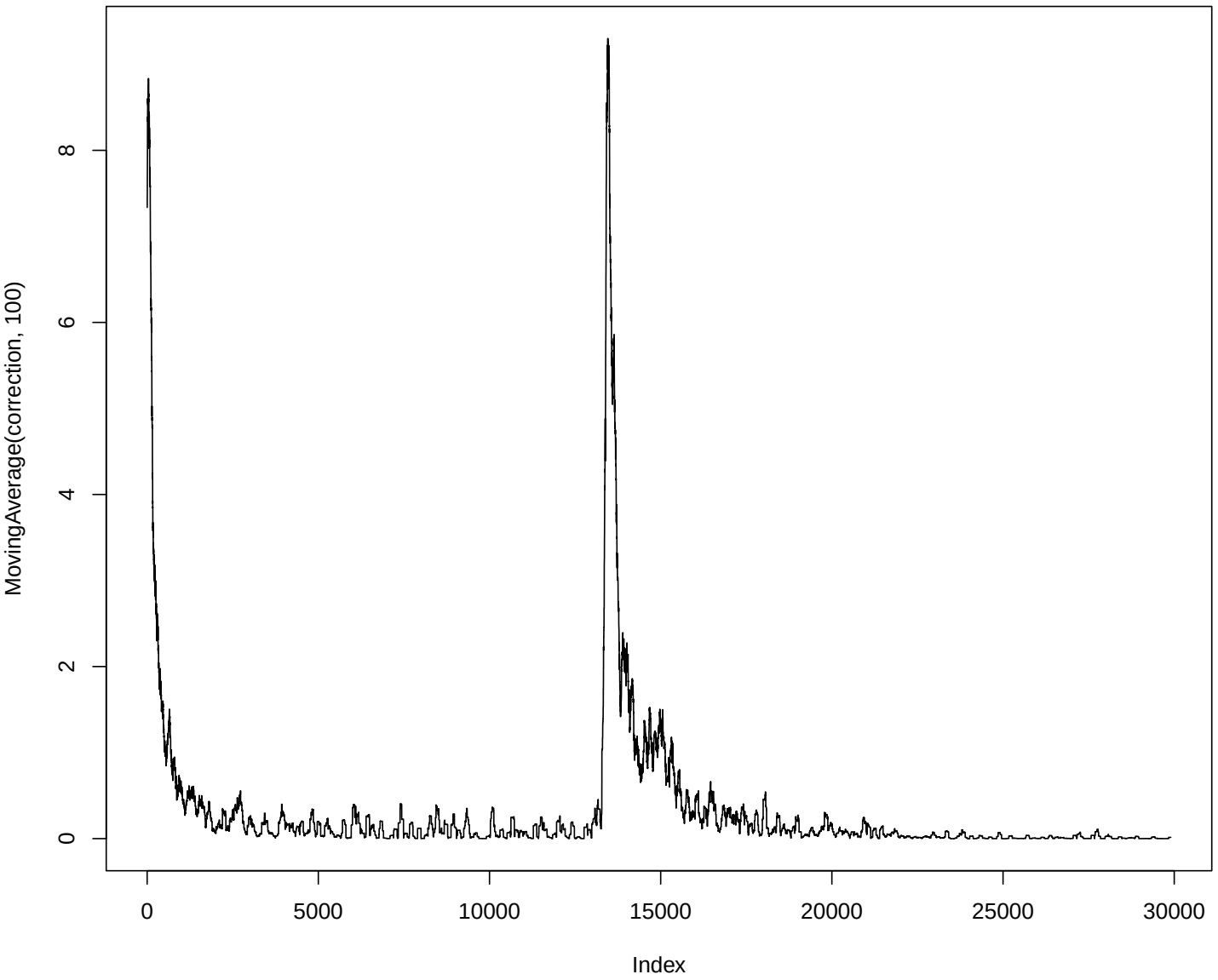

Q−table after 30000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.75 , beta = 0)
−1 −1 −1 −1 −1 −1 −1 −1
−1 −1 −1 −1 −1 −0.96 −0.65 −0.27
0.89 V 1.19 0.89 V 1.58 1.19 > 2.11 1.58 V 2.81 2.11 V 2.69 2.81 V 1.96 2.06 V 0.7 0.71 V 0.16
1.19 1.58 2.11 2.81 3.75 4.11 5.37 2.8
0.89 1.19 1.58 2.11 2.81 2.86 3.57 0.47
1.19 V 1.58 1.19 > 2.11 1.58 > 2.81 2.11 > 3.75 2.81 V 4.22 3.75 > 5.62 4.15 V 6.53 3.05 V 3.18
1.58 2.11 2.81 3.75 5 5.11 7.5 9.62
1.19 1.58 2.11 2.81 3.82 5.61
1.58 > 2.11 1.58 > 2.81 2.11 > 3.75 2.81 > 5 5 5 > 7.49 5.49 > 10 10
1.19 1.58 2.11 2.81 4 5.62
1.58 2.11 2.81 3.75 5 5.27 7.5 9.62
1.19 ^ 1.58 1.19 ^ 2.11 1.58 ^ 2.81 2.11 > 3.75 2.81 ^ 4.22 3.75 > 5.62 4.14 ^ 6.6 4 ^ 1.99
0.89 1.19 1.58 2.11 2.81 3.04 3.39 0.37
1.19 1.58 2.11 2.81 3.75 4.19 5.25 2.68
0.89 ^ 1.19 0.89 ^ 1.58 1.19 ^ 2.11 1.58 ^ 2.81 2.11 ^ 3.02 2.33 ^ 1.85 2.32 ^ 0.43 0.53 ^ 0.02
−1 −1 −1 −1 −1 −0.97 −0.72 −0.27
−1 −1 −1 −1 −1 −1 −1 −1
x
7
6
5
4
3
2
1
y
1 2 3 4 5 6 7 8
10.0
7.5
5.0
2.5


MovingAverage(reward, 100)
3.5
4.0
4.5
5.0
0
5000
10000
15000
20000
25000
30000
Index


MovingAverage(correction, 100)
0
1
2
3
4
0
5000
10000
15000
20000
25000
30000
Index


Q−table after 30000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.95 , beta = 0)
−1 −1 −1 −1 −1 −1 −1 −1
−1 −1 −1 −1 −1 −1 −1 −0.99
6.3 V 6.63 6.3 V 6.98 6.63 V 7.35 6.98 V 7.74 7.35 V 8.15 7.74 V 8.57 8.15 V 9.02 8.55 V 8.96
6.63 6.98 7.35 7.74 8.15 8.57 9.02 9.5
6.3 6.63 6.98 7.35 7.74 8.15 8.57 9.02
6.63 > 6.98 6.63 > 7.35 6.98 > 7.74 7.35 > 8.15 7.74 > 8.57 8.15 V 9.02 8.57 V 9.5 9.02 V 9.5
6.3 6.63 6.98 7.35 5 9.02 9.5 10
6.63 6.98 7.35 7.74 8.57 9.02
6.3 ^ 6.63 6.3 > 6.98 6.63 ^ 7.35 6.98 ^ 5 5 > 9.5 9.02 > 10 10
6.63 6.98 7.35 7.74 8.57 9.02
6.3 6.63 6.98 7.35 5 9.02 9.5 10
6.63 > 6.98 6.63 > 7.35 6.98 > 7.74 7.35 > 8.15 7.74 > 8.57 8.15 > 9.02 8.57 > 9.5 9.02 ^ 9.5
6.3 6.63 6.98 7.35 7.74 8.15 8.57 8.14
6.63 6.98 7.35 7.74 8.15 8.57 9.02 8.23
6.3 ^ 6.63 6.3 ^ 6.98 6.63 ^ 7.35 6.98 ^ 7.74 7.35 ^ 8.15 7.74 ^ 8.57 8.15 ^ 8.14 8.57 < 7.99
−1 −1 −1 −1 −1 −1 −1 −0.95
−1 −1 −1 −1 −1 −1 −1 −1
x
y
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7
10 9 8 7 6 5


MovingAverage(reward, 100)
4
5
6
7
8
0
5000
10000
15000
20000
25000
30000
Index



Increasing gamma makes future rewards count more, so the agent is more willing to take the longer route to the +10 instead of stopping at the +5. With gamma = 0.5 and 0.75 the agent goes for the 5, and with gamma = 0.95 it walks around the 5 to get to the 10.

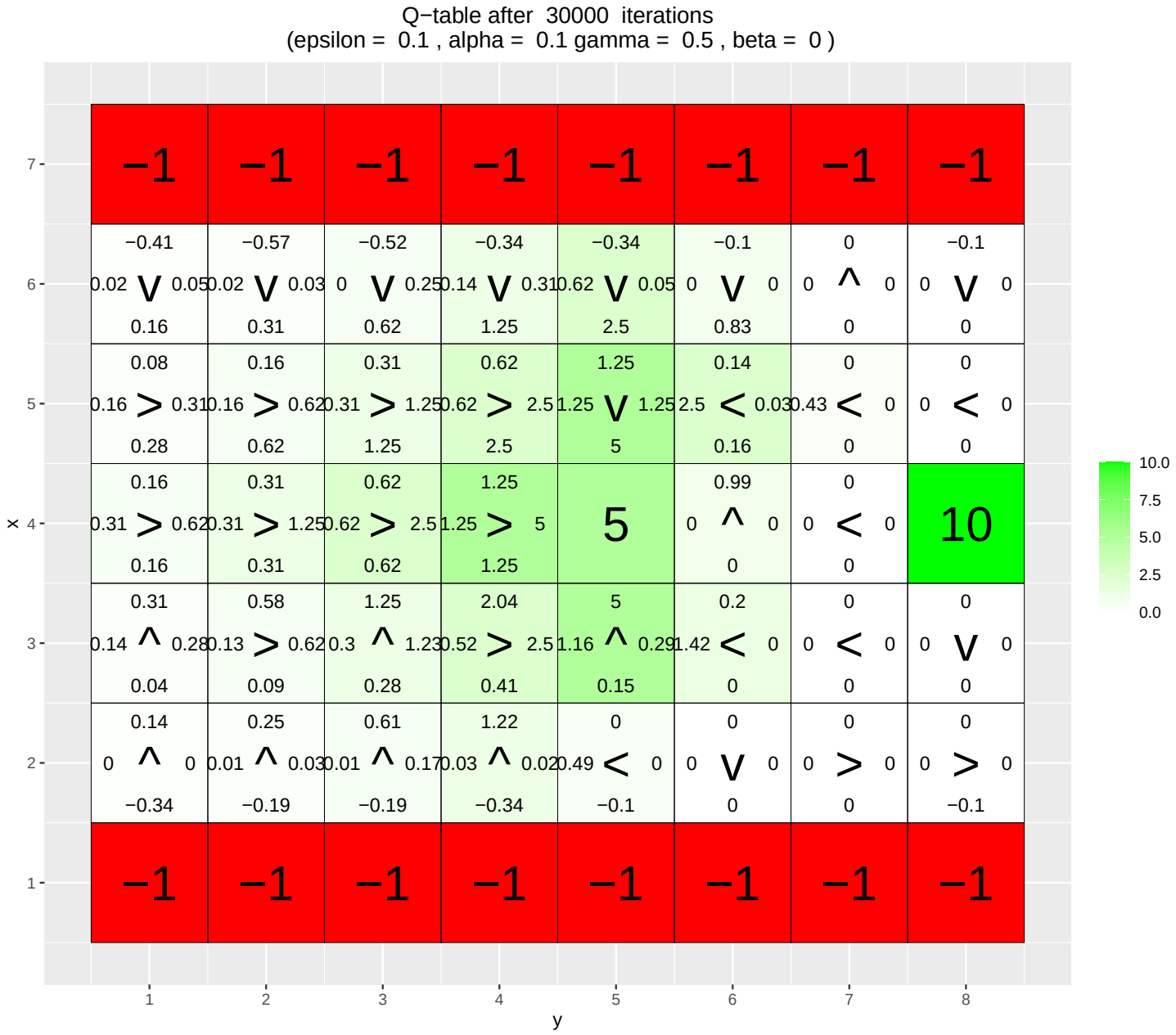
From the start state the 5 is 4 steps away and the 10 is 9 steps away (it has to walk around the 5), so the values are roughly $\gamma^3 \cdot 5$ vs $\gamma^8 \cdot 10$:

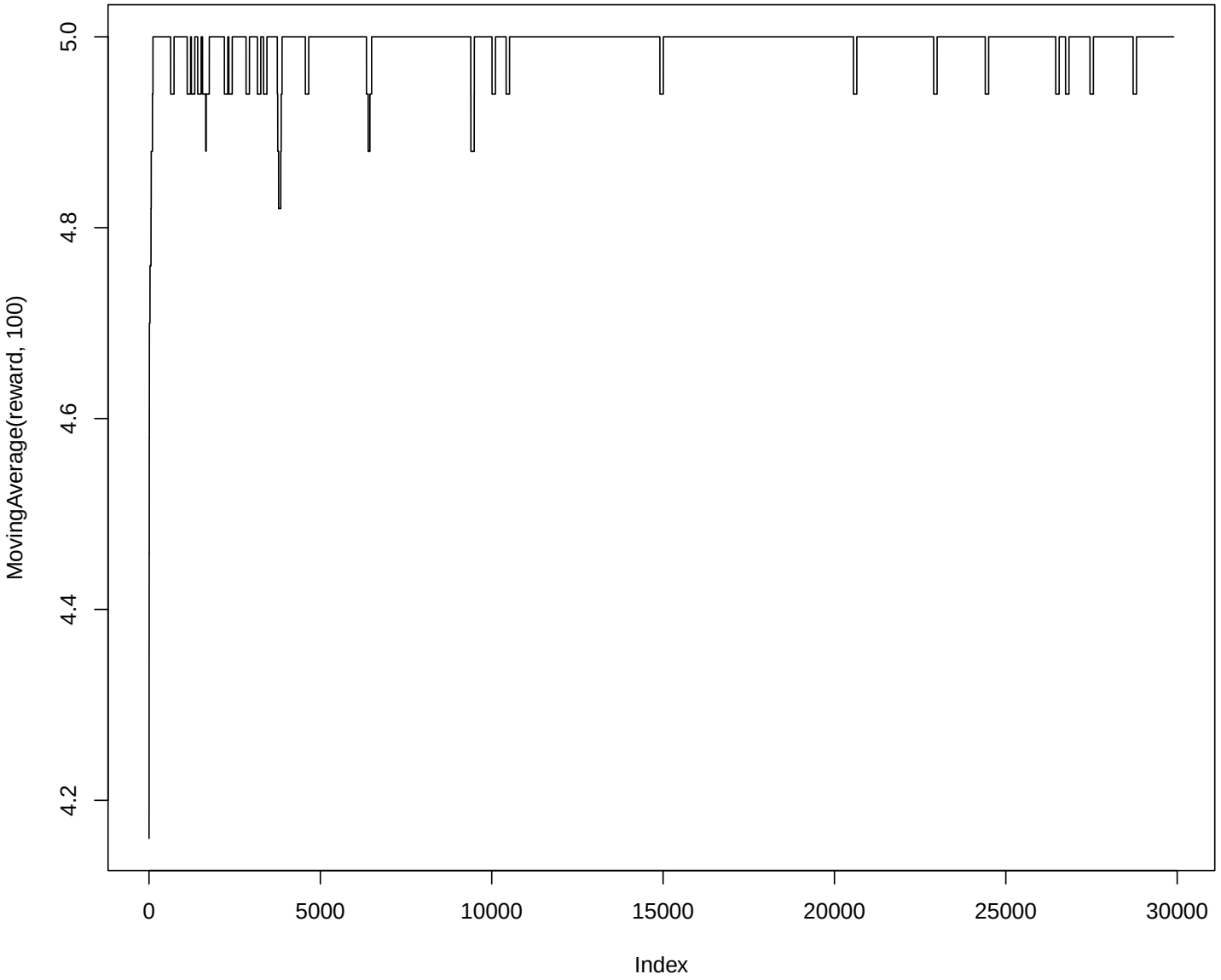
gamma	value of going to +5	value of going to +10
0.5	0.63	0.04
0.75	2.11	1.00
0.95	4.29	6.63

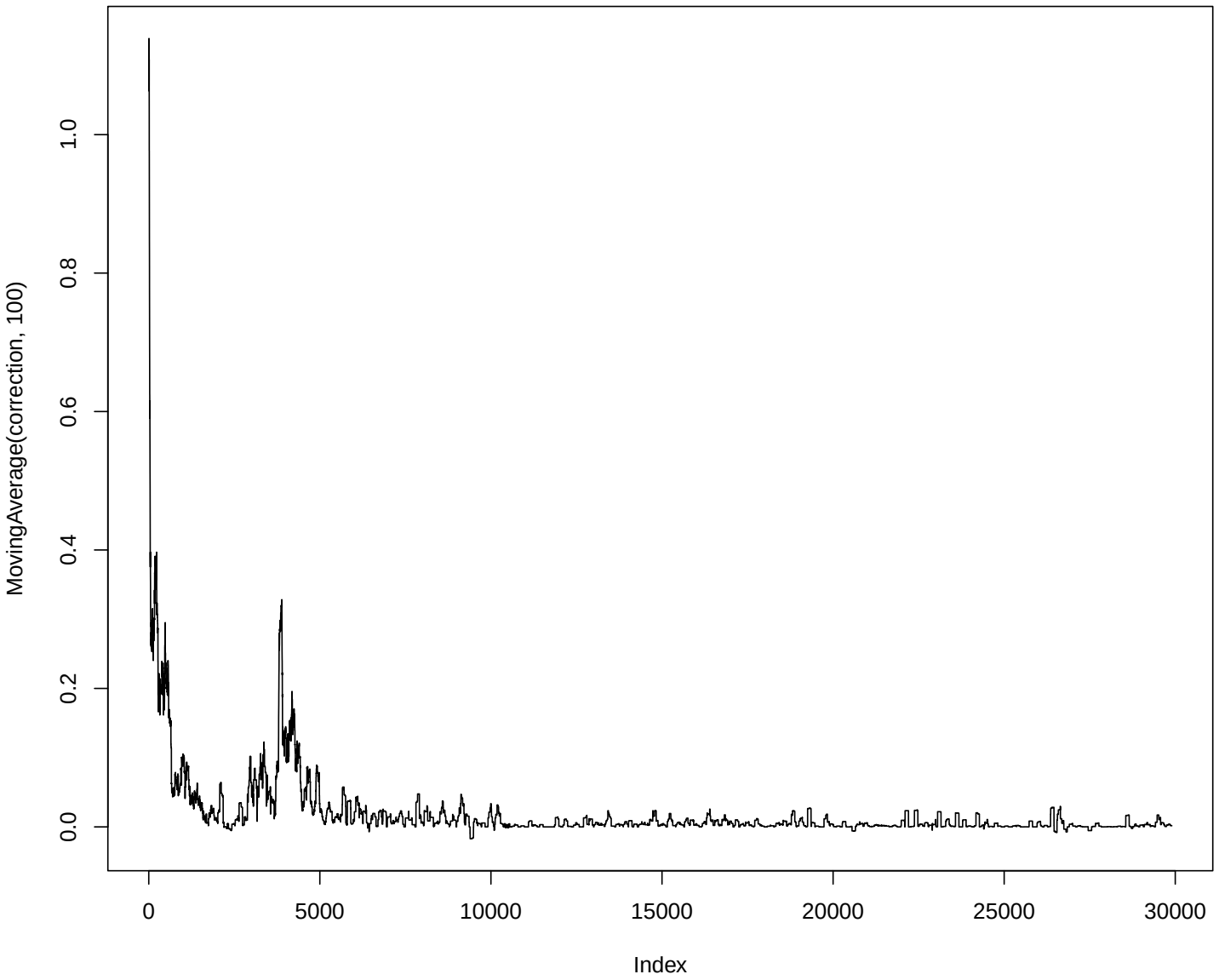
So the 10 is only worth the detour when gamma = 0.95. These numbers also show up in the Q-tables, the best action in the start state has value 0.62, 2.11 and 6.63 for the three gammas. With gamma 0.5 and 0.75 the states close to the 10 still point toward it, since from there the 10 is the better one.

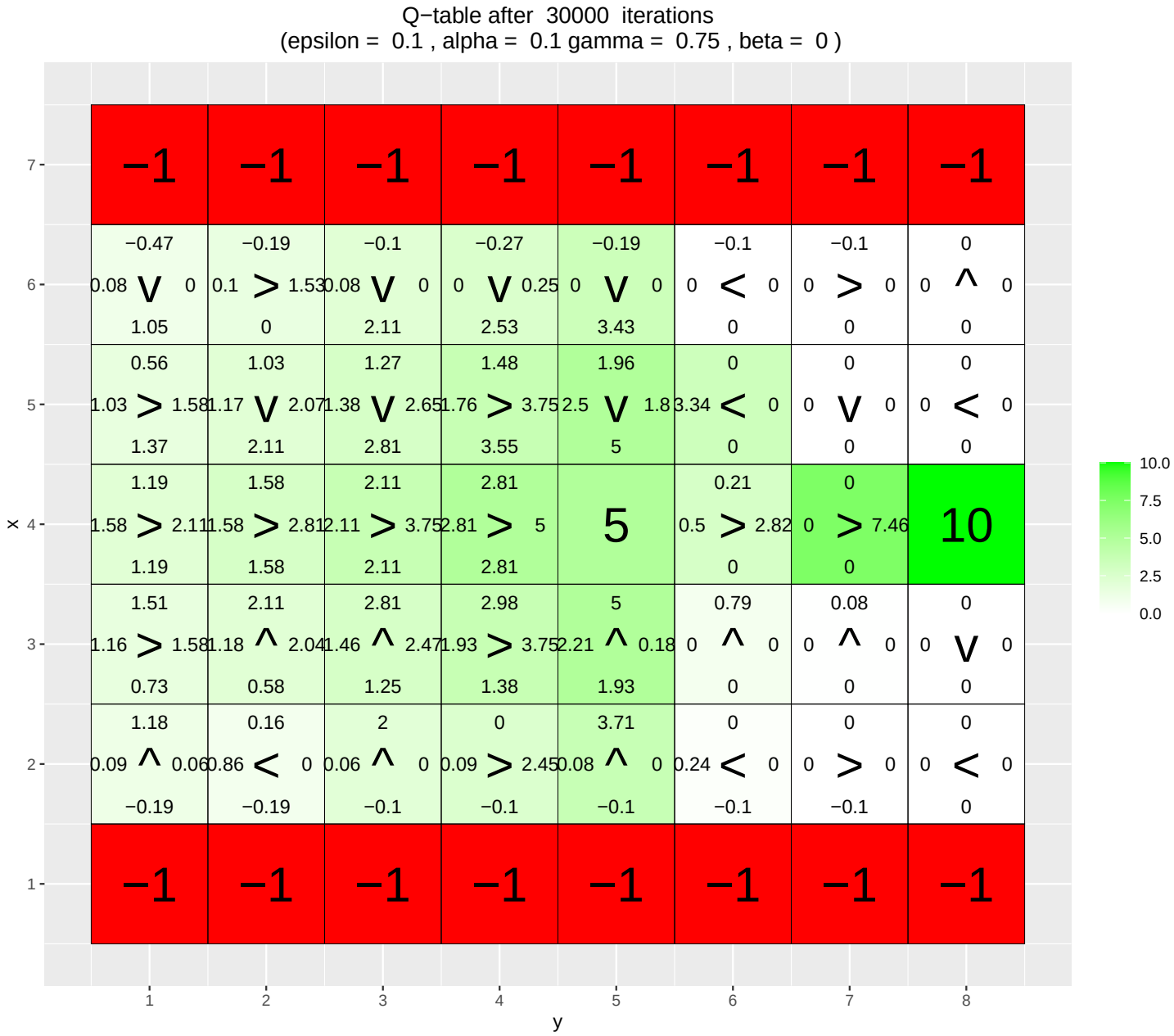
For gamma = 0.95 the reward plot shows a clear jump around episode 13 500, from about 4.5 to about 6.5. That's when the agent finds the route to the 10. The correction plot spikes at the same time, because the Q-values along the new route are off and it takes a few thousand episodes for them to settle.

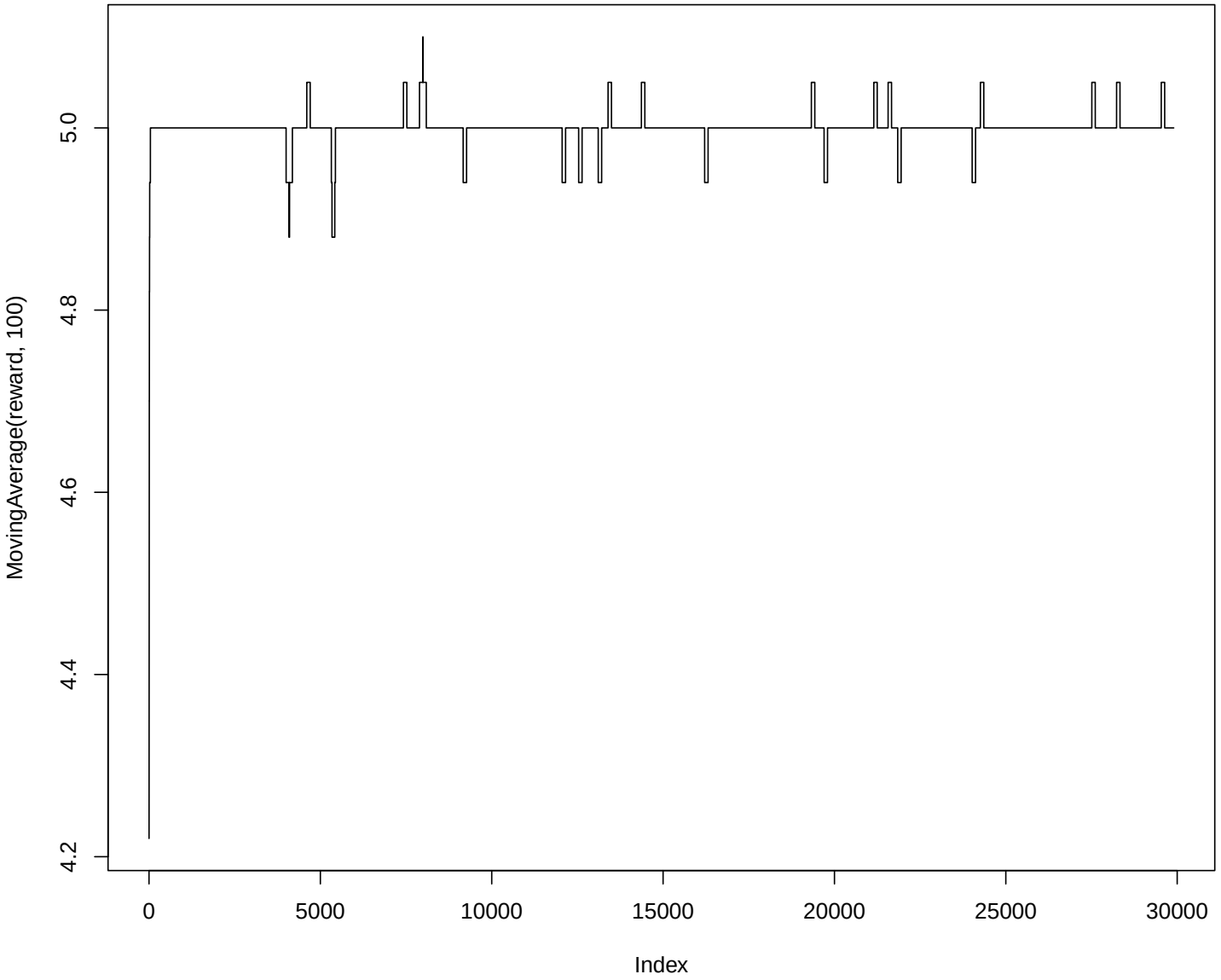
Effects of gamma with epsilon = 0.1

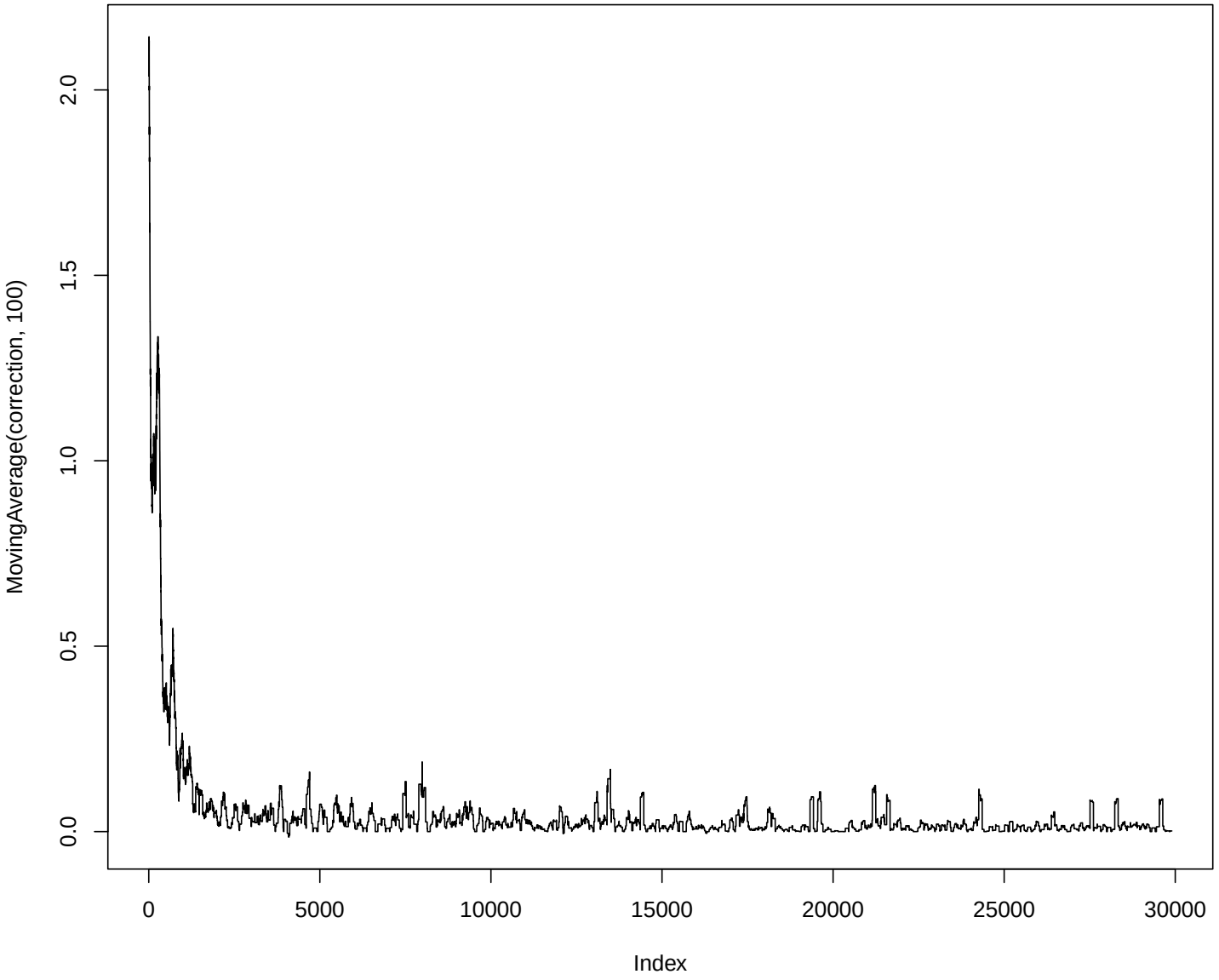


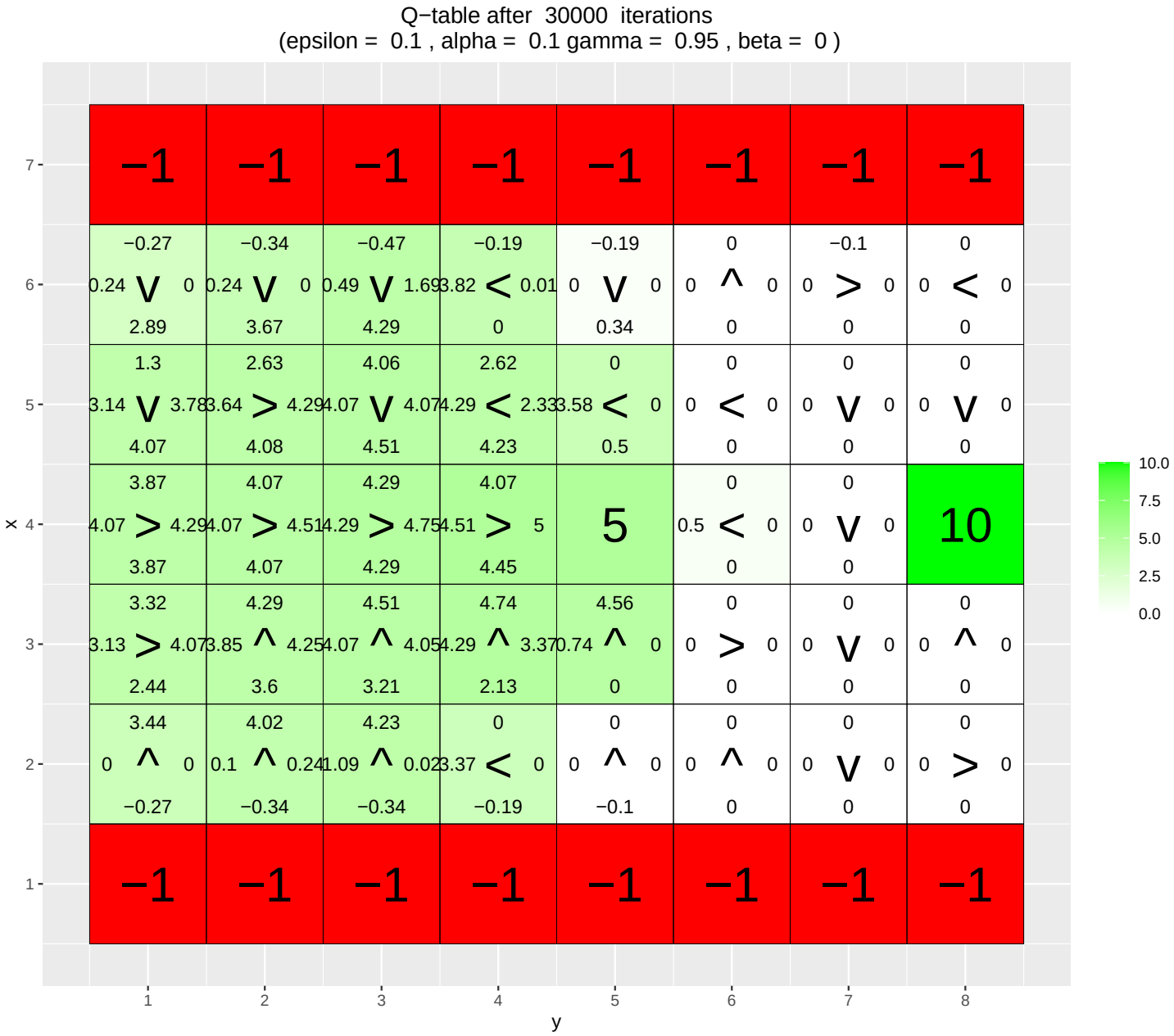

MovingAverage(reward, 100)
4.2
4.4
4.6
4.8
5.0
0
5000
10000
15000
20000
25000
30000
Index

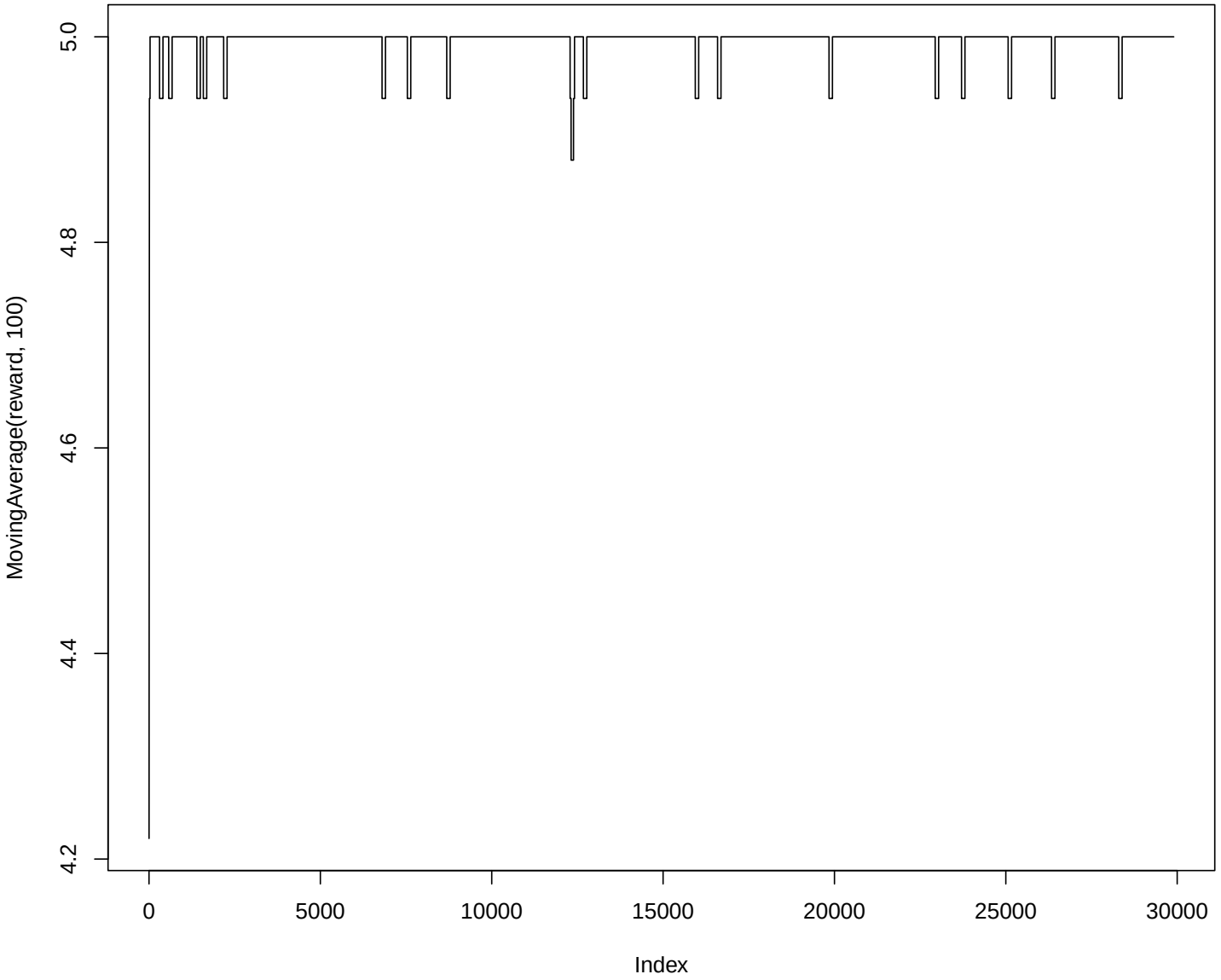

MovingAverage(correction, 100)
0.0
0.2
0.4
0.6
0.8
1.0
0
5000
10000
15000
20000
25000
30000
Index

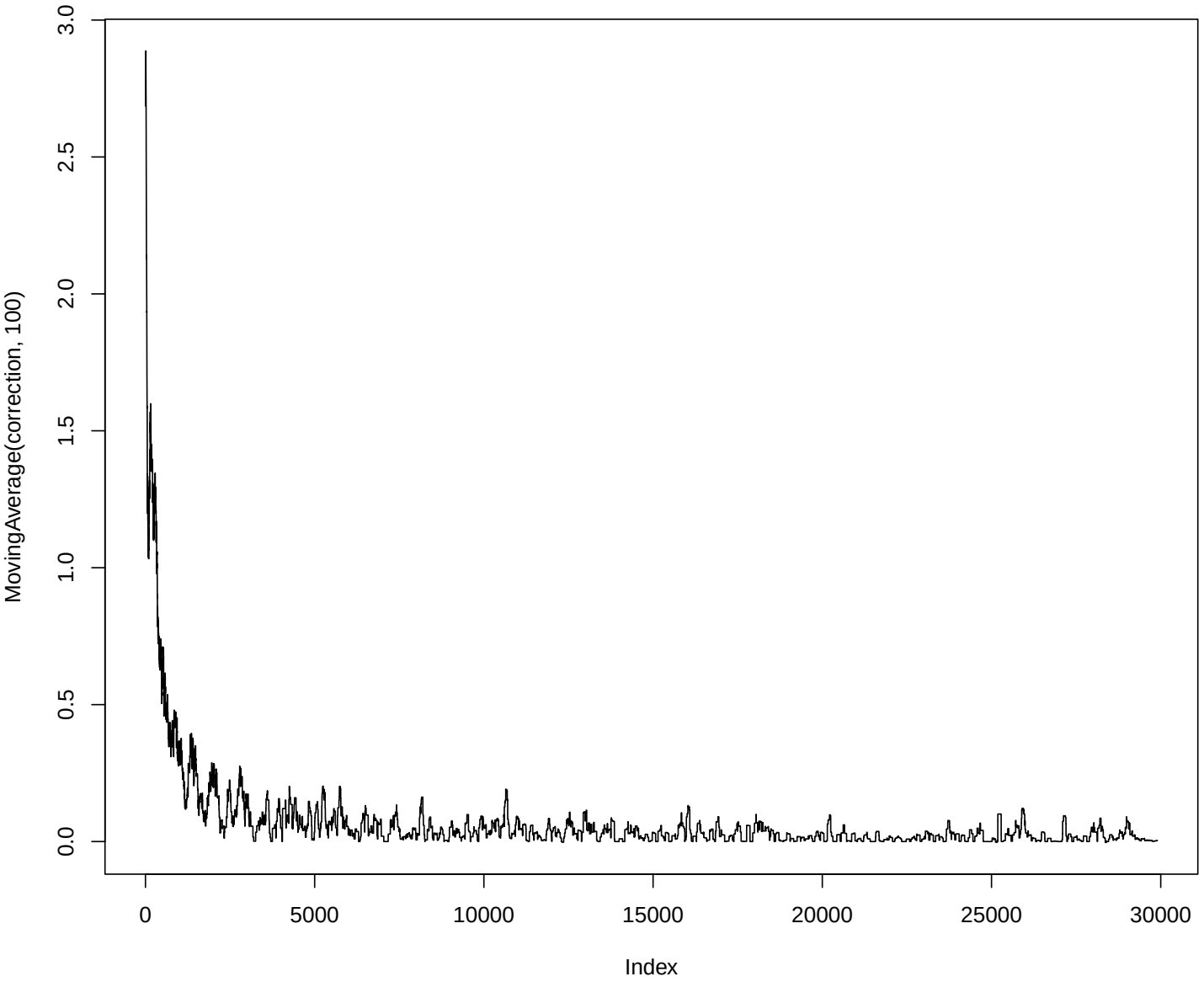

Q−table after 30000 iterations
(epsilon = 0.1 , alpha = 0.1 gamma = 0.75 , beta = 0)
−1 −1 −1 −1 −1 −1 −1 −1
−0.47 −0.19 −0.1 −0.27 −0.19 −0.1 −0.1 0
0.08 V 0 0.1 > 1.53 0.08 V 0 0 V 0.25 0 V 0 0 < 0 0 > 0 0 ^ 0
1.05 0 2.11 2.53 3.43 0 0 0
0.56 1.03 1.27 1.48 1.96 0 0 0
1.03 > 1.58 1.17 V 2.07 1.38 V 2.65 1.76 > 3.75 2.5 V 1.8 3.34 < 0 0 V 0 0 < 0
1.37 2.11 2.81 3.55 5 0 0 0
1.19 1.58 2.11 2.81 0.21 0
1.58 > 2.11 1.58 > 2.81 2.11 > 3.75 2.81 > 5 5 0.5 > 2.82 0 > 7.46 10
1.19 1.58 2.11 2.81 0 0
1.51 2.11 2.81 2.98 5 0.79 0.08 0
1.16 > 1.58 1.18 ^ 2.04 1.46 ^ 2.47 1.93 > 3.75 2.21 ^ 0.18 0 ^ 0 0 ^ 0 0 V 0
0.73 0.58 1.25 1.38 1.93 0 0 0
1.18 0.16 2 0 3.71 0 0 0
0.09 ^ 0.06 0.86 < 0 0.06 ^ 0 0.09 > 2.45 0.08 ^ 0 0.24 < 0 0 > 0 0 < 0
−0.19 −0.19 −0.1 −0.1 −0.1 −0.1 −0.1 0
−1 −1 −1 −1 −1 −1 −1 −1
x
y
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7
10.0
7.5
5.0
2.5
0.0


MovingAverage(reward, 100)
4.2
4.4
4.6
4.8
5.0
0
5000
10000
15000
20000
25000
30000
Index


MovingAverage(correction, 100)
0.0
0.5
1.0
1.5
2.0
0
5000
10000
15000
20000
25000
30000
Index


Q−table after 30000 iterations
(epsilon = 0.1 , alpha = 0.1 gamma = 0.95 , beta = 0)
−1 −1 −1 −1 −1 −1 −1 −1
−0.27 −0.34 −0.47 −0.19 −0.19 0 −0.1 0
0.24 V 0 0.24 V 0 0.49 V 1.69 3.82 < 0.01 0 V 0 0 ^ 0 0 > 0 0 < 0
2.89 3.67 4.29 0 0.34 0 0 0
1.3 2.63 4.06 2.62 0 0 0 0
3.14 V 3.78 3.64 > 4.29 4.07 V 4.07 4.29 < 2.33 3.58 < 0 0 < 0 0 V 0 0 V 0
4.07 4.08 4.51 4.23 0.5 0 0 0
3.87 4.07 4.29 4.07 0 0 0
4.07 > 4.29 4.07 > 4.51 4.29 > 4.75 4.51 > 5 5 0.5 < 0 0 V 0 10
3.87 4.07 4.29 4.45 0 0 0
3.32 4.29 4.51 4.74 4.56 0 0 0
3.13 > 4.07 3.85 ^ 4.25 4.07 ^ 4.05 4.29 ^ 3.37 0.74 ^ 0 0 > 0 0 V 0 0 ^ 0
2.44 3.6 3.21 2.13 0 0 0 0
3.44 4.02 4.23 0 0 0 0 0
0 ^ 0 0.1 ^ 0.24 1.09 ^ 0.02 3.37 < 0 0 ^ 0 0 ^ 0 0 V 0 0 > 0
−0.27 −0.34 −0.34 −0.19 −0.1 0 0 0
−1 −1 −1 −1 −1 −1 −1 −1
x
y
1 2 3 4 5 6 7 8
10.0
7.5
5.0
2.5
0.0


MovingAverage(reward, 100)
5.0
4.8
4.6
4.4
4.2
0
5000
10000
15000
20000
25000
30000
Index

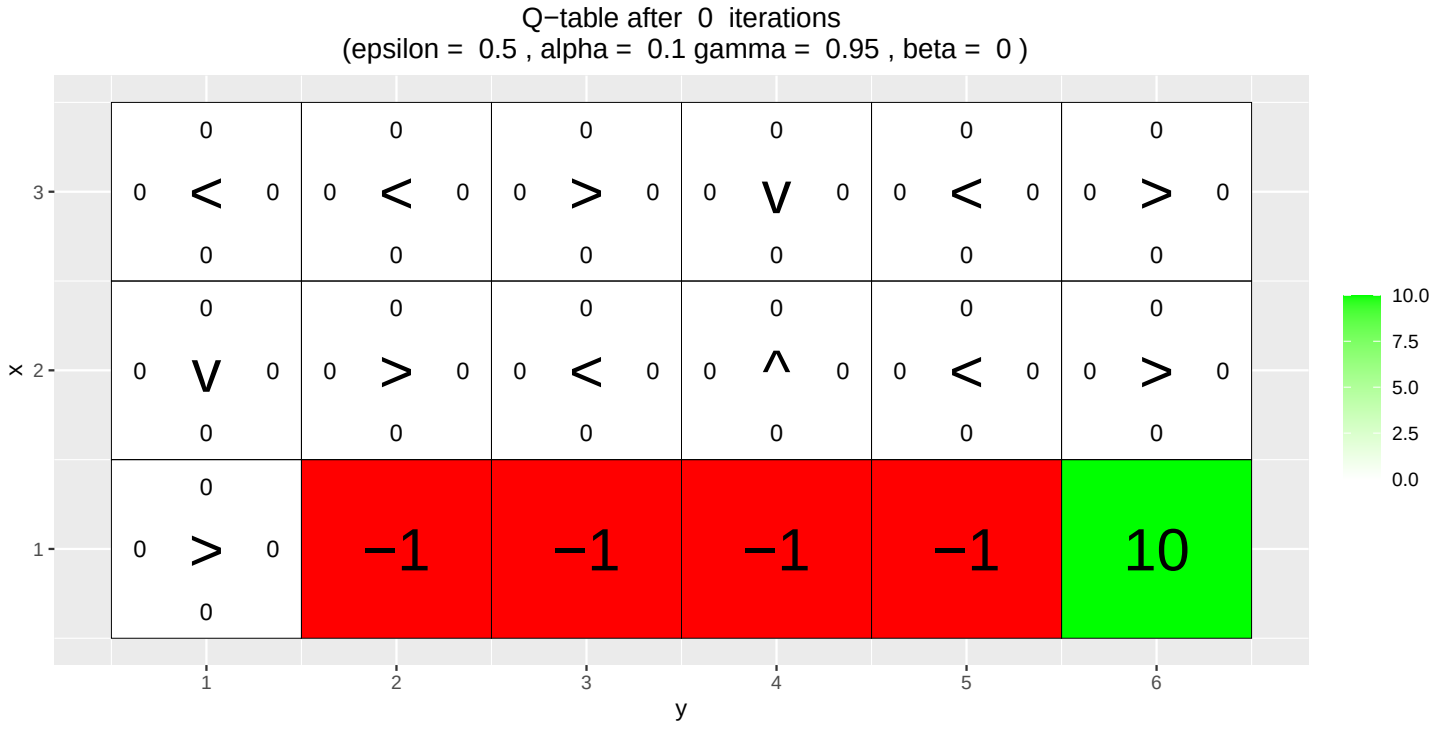


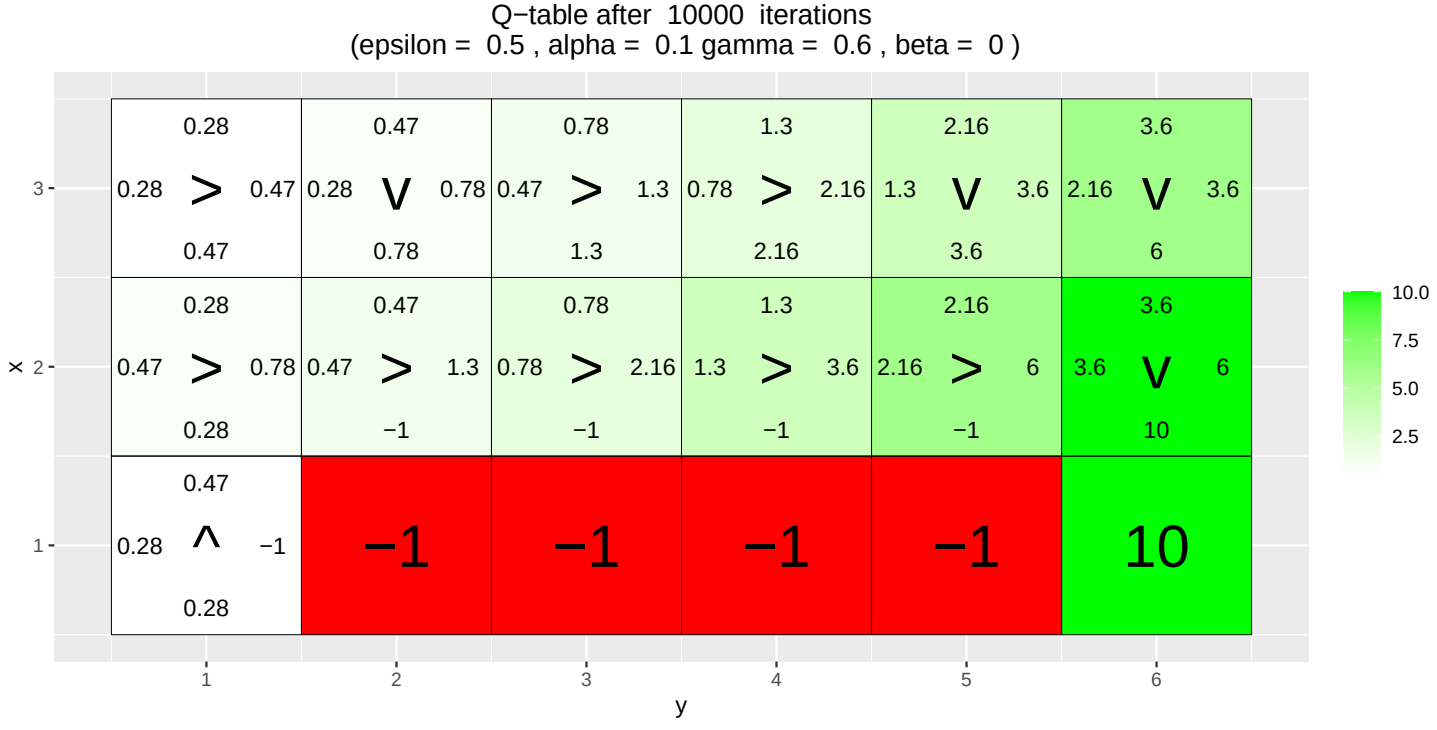
With epsilon = 0.1 the agent only acts randomly 10% of the time. To find the 10 it has to get around the 5 by chance, which takes a lot of random moves in a row, so it almost never happens. The agent learns the route to the 5 early and keeps taking it, for all three gammas. You can see this as most states to the right of the 5 are still 0, meaning they were rarely visited.

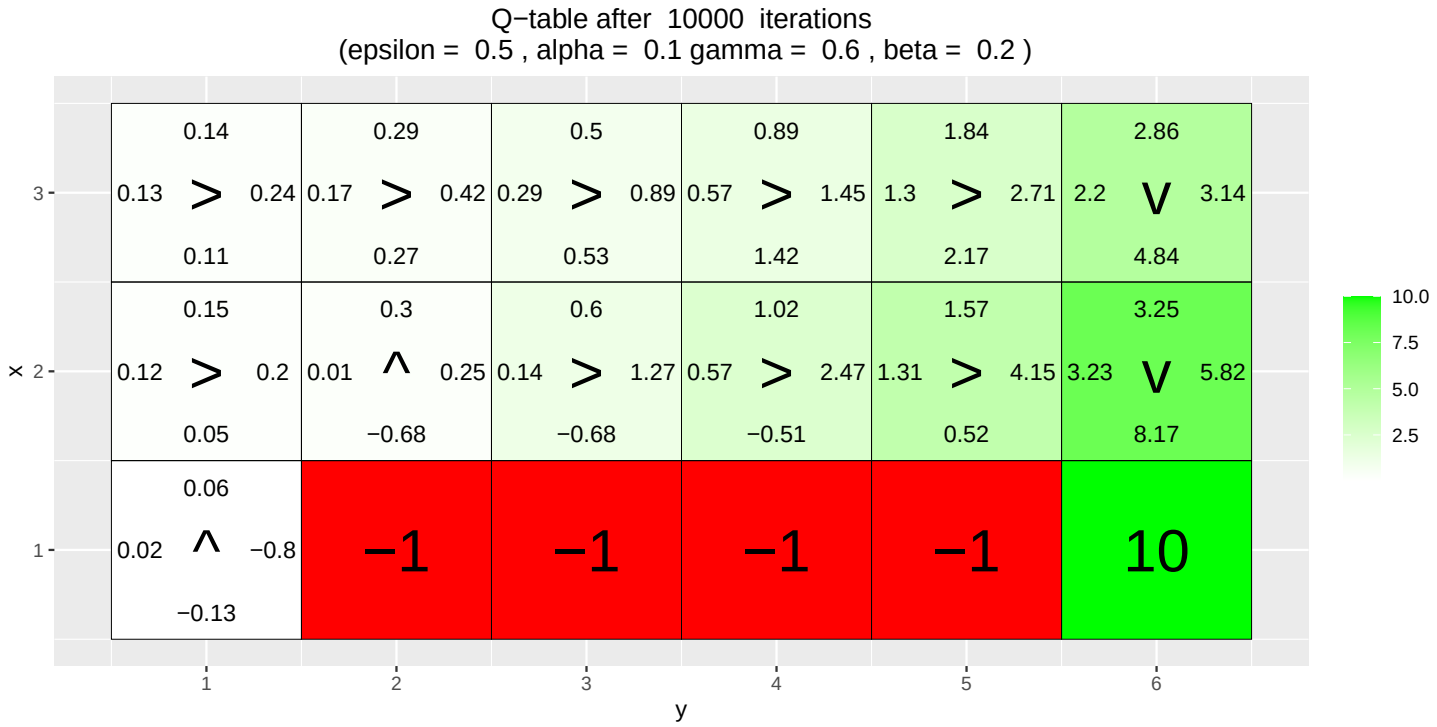
For gamma = 0.5 and 0.75 this doesn't matter much, since the 5 is the better choice anyway. But for gamma = 0.95, going to the 10 would be worth 6.63 from the start, but the agent never explores enough to find out, so it settles for the 5 (4.29).

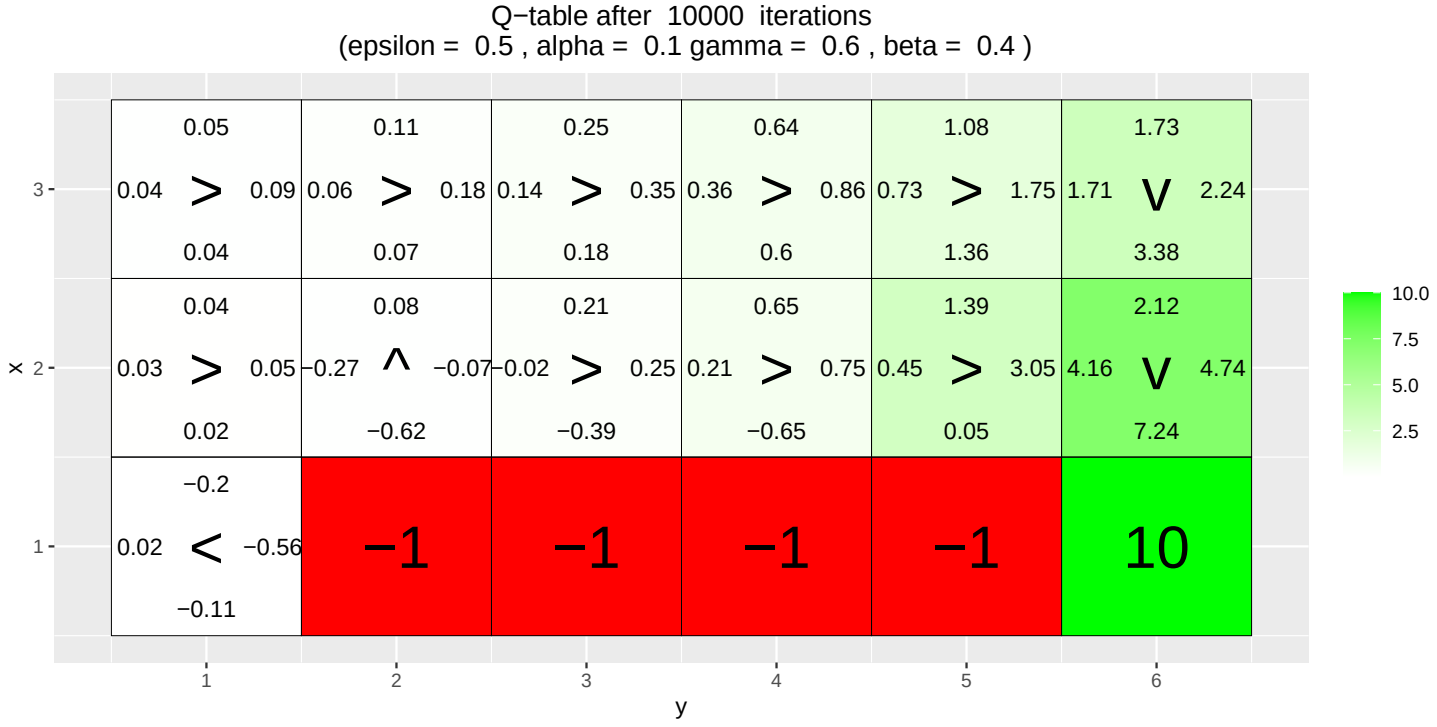
1.3 Environment C: effect of beta

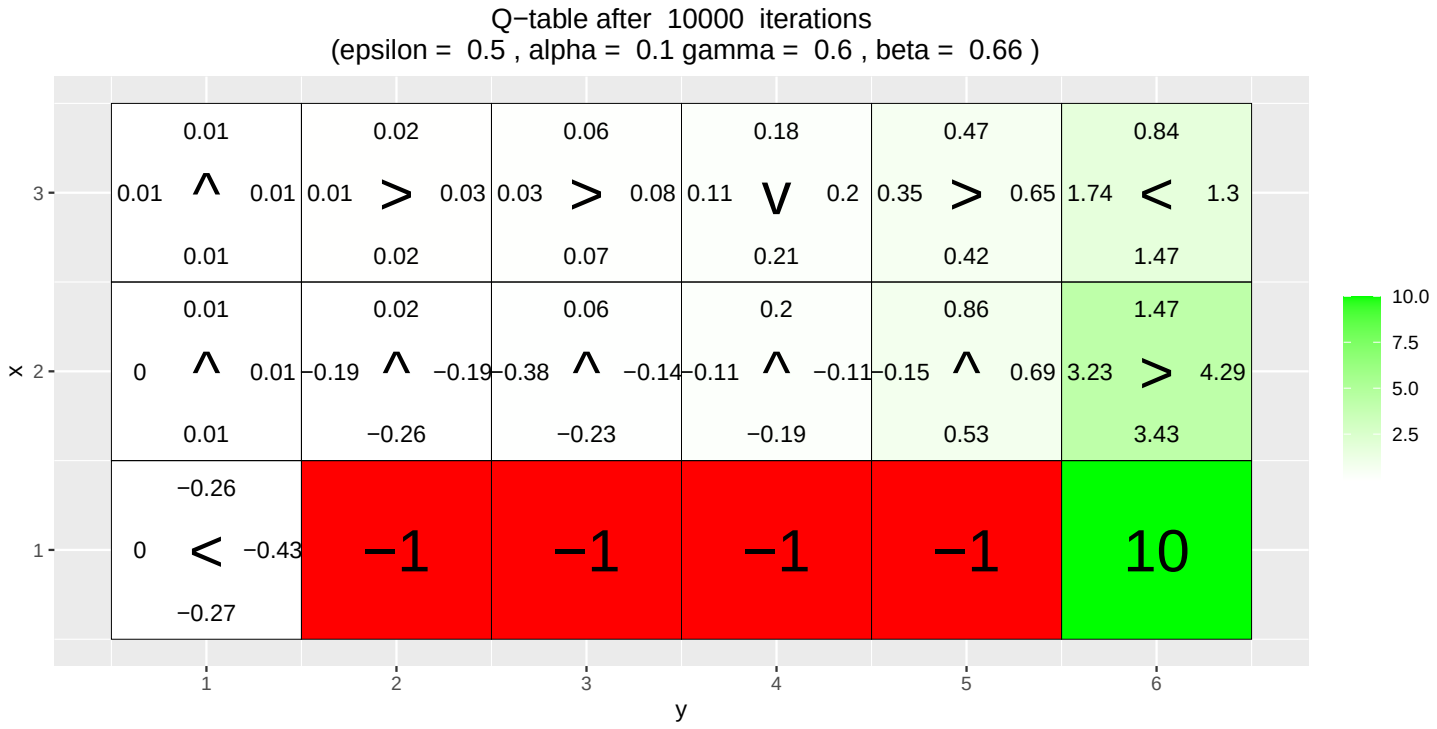
Environment C places a +10 reward beyond a sequence of negative-reward states. The experiment compares beta values 0, 0.2, 0.4 and 0.66, with epsilon 0.5 and gamma 0.6. Beta controls the probability of slipping either right or left of current state. Each side-slip has probability beta/2, while the intended move has probability 1-beta. As beta increases, movement becomes less predictable and reaching the goal becomes harder.






Q−table after 10000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.6 , beta = 0.2)
0.14
0.13 > 0.24
0.11
0.29
0.17 > 0.42
0.27
0.5
0.29 > 0.89
0.53
0.89
0.57 > 1.45
1.42
1.84
1.3 > 2.71
2.17
2.86
2.2 V 3.14
4.84
0.15
0.12 > 0.2
0.05
0.3
0.01 ^ 0.25
−0.68
0.6
0.14 > 1.27
−0.68
1.02
0.57 > 2.47
−0.51
1.57
1.31 > 4.15
0.52
3.25
3.23 V 5.82
8.17
0.06
0.02 ^ −0.8
−0.13
−1
−1
−1
−1
10
x
y
10.0
7.5
5.0
2.5


Q−table after 10000 iterations
(epsilon = 0.5 , alpha = 0.1 gamma = 0.6 , beta = 0.4)
0.05
0.04 > 0.09
0.04
0.11
0.06 > 0.18
0.07
0.25
0.14 > 0.35
0.18
0.64
0.36 > 0.86
0.6
1.08
0.73 > 1.75
1.36
1.73
1.71 V 2.24
3.38
0.04
0.03 > 0.05
0.02
0.08
−0.27 ^ −0.07
−0.62
0.21
−0.02 > 0.25
−0.39
0.65
0.21 > 0.75
−0.65
1.39
0.45 > 3.05
0.05
2.12
4.16 V 4.74
7.24
−0.2
0.02 < −0.56
−0.11
−1
−1
−1
−1
10
x
y
10.0
7.5
5.0
2.5



Observations

With beta = 0 there's no slipping, so the agent just takes the shortest path along the middle row, right next to the -1s. There's no risk in doing that. When we increase beta, the agent starts keeping its distance from the -1s and prefers the top row, since from there it can't slip into them. It happens gradually. With beta = 0.66 the whole middle row from (2, 1) to (2, 5) points up. The Q-values also get lower as beta increases, since the paths get longer and riskier.

An interesting thing is that in some states the greedy policy says to walk into the wall. Take the start state (1, 1) with beta = 0.4 and 0.66: if the agent walks left into the wall, it either stays where it is or slips up into (2, 1) (slipping down is also a wall). So it can never slip right into the -1 at (1, 2). Same thing at (2, 6) with beta = 0.66, where it walks right into the wall and either stays, slips up to (3, 6), or slips down into the goal.

With beta = 0.66 the intended move only happens with probability 0.34, which is about the same as each slip (0.33). So the action the agent picks basically only decides which three directions are possible, not which one actually happens. That's why "up" is chosen in the whole middle row, then the agent can end up going up, left or right, but never down into the -1.

2. REINFORCE

The REINFORCE implementation and its results are provided in `RL_Lab3_Colab.ipynb`.

The policy is a small neural network (two hidden layers with 32 units) that takes the four coordinates $(x, y, x_{goal}, y_{goal})$ as input and outputs a probability for each of the four actions.

2.1 Environment D

Eight goal positions are used for training and the other eight for validation.

Has the agent learned a good policy? Why or why not?

Yes, mostly. In the validation plots almost every arrow points toward the goal, even though the agent has never seen any of these goals during training. That's because the policy is a neural network and not a lookup table. It learns a function of the coordinates, something like "move in the direction of goal minus position", and the same weights are shared across all inputs.

But it's not perfect. For goal (4, 2), the state (4, 1) right next to it points down instead of right, and (3, 1) points back up. So following the arrows the agent would go back and forth between those two states. This happens because the reward is only positive so every action in a successful episode gets reinforced, even detours. Episodes where the agent doesn't reach the goal give no signal, so bad actions never get negatively rewarded.

Could Q-learning solve this task?

Yes, if we include the goal as part of the state. But tabular Q-learning can't generalize. Every (state, goal) combination has its own entries in the table, so the Q-values for the 8 validation goals would never be updated and stay at 0, which gives a random policy there. We would have to train on all 16 goals.

The generalization comes from using a neural network not from REINFORCE itself. Q-learning with a neural network instead of a table would be able to generalize in the same way.

2.2 Environment E

Training goals are all in the top row, while the three validation goals (3, 4), (2, 3) and (1, 1) are in the rows below.

Has the agent learned a good policy? Why or why not?

No. Looking at the validation plots the network only learned to move toward the right column, but not toward the right row. During training the goal row was always 4, so the network never saw that input change and had no way to learn that it matters. And "go up" was always a good move during training, so that's what it does everywhere.

If the results obtained for environments D and E differ, explain why.

They differ a lot. In D the training goals cover every row and column, so the validation goals are new combinations of values the network has already seen. In E the goal row never changes during training, so the validation goals have row values the network has never seen. Generalization only works if the training data has enough variation.

Appendix A: R Code

The Q-learning code is listed below, grouped by environment.

A.1 Q-learning functions

```r
# By Jose M. Peña and Joel Oskarsson.
# For teaching purposes.
# jose.m.pena@liu.se.

##########################################################################################################
# Q-learning
##########################################################################################################

# install.packages("ggplot2")
# install.packages("vctrs")
library(ggplot2)

arrows <- c("^", ">", "v", "<")
action_deltas <- list(c(1,0), # up
                      c(0,1), # right
                      c(-1,0), # down
                      c(0,-1)) # left

vis_environment <- function(iterations=0, epsilon = 0.5, alpha = 0.1, gamma = 0.95, beta = 0){

  # Visualize an environment with rewards.
  # Q-values for all actions are displayed on the edges of each tile.
  # The (greedy) policy for each state is also displayed.
  #
  # Args:
  #   iterations, epsilon, alpha, gamma, beta (optional): for the figure title.
  #   reward_map (global variable): a HxW array containing the reward given at each state.
  #   q_table (global variable): a HxWx4 array containing Q-values for each state-action pair.
  #   H, W (global variables): environment dimensions.

  df <- expand.grid(x=1:H,y=1:W)
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,1],NA),df$x,df$y)
```

```
  df$val1 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,2],NA),df$x,df$y)
  df$val2 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,3],NA),df$x,df$y)
  df$val3 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,q_table[x,y,4],NA),df$x,df$y)
  df$val4 <- as.vector(round(foo, 2))
  foo <- mapply(function(x,y)
    ifelse(reward_map[x,y] == 0,arrows[GreedyPolicy(x,y)],reward_map[x,y]),df$x,df$y)
  df$val5 <- as.vector(foo)
  foo <- mapply(function(x,y) ifelse(reward_map[x,y] == 0,max(q_table[x,y,]),
                                     ifelse(reward_map[x,y]<0,NA,reward_map[x,y])),df$x,df$y)
  df$val6 <- as.vector(foo)

  print(ggplot(df,aes(x = y,y = x)) +
          scale_fill_gradient(low = "white", high = "green", na.value = "red", name = "") +
          geom_tile(aes(fill=val6)) +
          geom_text(aes(label = val1),size = 4,nudge_y = .35,na.rm = TRUE) +
          geom_text(aes(label = val2),size = 4,nudge_x = .35,na.rm = TRUE) +
          geom_text(aes(label = val3),size = 4,nudge_y = -.35,na.rm = TRUE) +
          geom_text(aes(label = val4),size = 4,nudge_x = -.35,na.rm = TRUE) +
          geom_text(aes(label = val5),size = 10) +
          geom_tile(fill = 'transparent', colour = 'black') +
          ggtitle(paste("Q-table after ",iterations," iterations\n",
                        "(epsilon = ",epsilon,", alpha = ",alpha,"gamma = ",gamma,", beta = ",beta,")")) +
          theme(plot.title = element_text(hjust = 0.5)) +
          scale_x_continuous(breaks = c(1:W),labels = c(1:W)) +
          scale_y_continuous(breaks = c(1:H),labels = c(1:H)))

}

GreedyPolicy <- function(x, y){

  # Get a greedy action for state (x,y) from q_table.
  #
  # Args:
  #   x, y: state coordinates.
  #   q_table (global variable): a HxWx4 array containing Q-values for each state-action pair.
  #
  # Returns:
  #   An action, i.e. integer in {1,2,3,4}.

  # q_values for each action in given state
  q_values <- q_table[x, y, ]
  # Action(s) which has highest q_value
  best_actions <- which(q_values == max(q_values))
  # Select action at random from all greedy ties
  action <- best_actions[sample.int(length(best_actions), 1)]

}

EpsilonGreedyPolicy <- function(x, y, epsilon){

  # Get an epsilon-greedy action for state (x,y) from q_table.
  #
  # Args:
  #   x, y: state coordinates.
  #   epsilon: probability of acting randomly.
  #
```

```
  # Returns:
  #   An action, i.e. integer in {1,2,3,4}.

  # Explore with probability epsilon, else choose a greedy action.
  if(runif(1) < epsilon) {
    # Random action
    action <- sample.int(4, 1)
  } else {
    # Greedy policy
    action <- GreedyPolicy(x, y)
  }
  return(action)
}

transition_model <- function(x, y, action, beta){

  # Computes the new state after given action is taken. The agent will follow the action
  # with probability (1-beta) and slip to the right or left with probability beta/2 each.
  #
  # Args:
  #   x, y: state coordinates.
  #   action: which action the agent takes (in {1,2,3,4}).
  #   beta: probability of the agent slipping to the side when trying to move.
  #   H, W (global variables): environment dimensions.
  #
  # Returns:
  #   The new state after the action has been taken.

  delta <- sample(-1:1, size = 1, prob = c(0.5*beta,1-beta,0.5*beta))
  final_action <- ((action + delta + 3) %% 4) + 1
  foo <- c(x,y) + unlist(action_deltas[final_action])
  foo <- pmax(c(1,1),pmin(foo,c(H,W)))

  return (foo)
}

q_learning <- function(start_state, epsilon = 0.5, alpha = 0.1, gamma = 0.95,
                       beta = 0){

  # Perform one episode of Q-learning. The agent should move around in the
  # environment using the given transition model and update the Q-table.
  # The episode ends when the agent reaches a terminal state.
  #
  # Args:
  #   start_state: array with two entries, describing the starting position of the agent.
  #   epsilon (optional): probability of acting randomly.
  #   alpha (optional): learning rate.
  #   gamma (optional): discount factor.
  #   beta (optional): slipping factor.
  #   reward_map (global variable): a HxW array containing the reward given at each state.
  #   q_table (global variable): a HxWx4 array containing Q-values for each state-action pair.
  #
  # Returns:
  #   reward: reward received in the episode.
  #   correction: sum of the temporal difference correction terms over the episode.
  #   q_table (global variable): Recall that R passes arguments by value. So, q_table being
  #   a global variable can be modified with the superassigment operator <<-.

  # Your code here.
```

```
  state <- start_state
  episode_correction <- 0
  repeat{
    # Follow policy, execute action, get reward.
    action <- EpsilonGreedyPolicy(state[1], state[2], epsilon)
    new_state <- transition_model(state[1], state[2], action, beta)
    reward <- reward_map[new_state[1], new_state[2]]

    # q_value of current  state-action pair
    curr_value <- q_table[state[1], state[2], action]
    # q_value of best action in new state
    next_value <- max(q_table[new_state[1], new_state[2], ])
    # the transition "error" based on the reward plus the best estimated future value
    delta <- reward + gamma * next_value - curr_value

    # Q-table update by adding product of "error" and alpha.
    q_table[state[1], state[2], action] <<-
      q_table[state[1], state[2], action] + alpha * delta

    # accumulate the TD correction term R + gamma*max_a Q(S',a) - Q(S,A) (unscaled by alpha)
    episode_correction <- episode_correction + delta
    # update state
    state <- new_state

    if(reward!=0)
      # End episode.
      return (c(reward,episode_correction))
  }

}
```

A.2 Environment A

```
#########################################################################################################
# Q-Learning Environments
#########################################################################################################

# Environment A (learning)

set.seed(12345)

H <- 5
W <- 7

reward_map <- matrix(0, nrow = H, ncol = W)
reward_map[3,6] <- 10
reward_map[2:4,3] <- -1

q_table <- array(0,dim = c(H,W,4))

vis_environment()

for(i in 1:10000){
  foo <- q_learning(start_state = c(3,1))

  if(any(i==c(10,100,1000,10000)))
    vis_environment(i)
}
```

A.3 Environment B: epsilon and gamma

```
# Environment B (the effect of epsilon and gamma)

set.seed(12345)

H <- 7
W <- 8

reward_map <- matrix(0, nrow = H, ncol = W)
reward_map[1,] <- -1
reward_map[7,] <- -1
reward_map[4,5] <- 5
reward_map[4,8] <- 10

q_table <- array(0,dim = c(H,W,4))

vis_environment()

MovingAverage <- function(x, n){

  cx <- c(0,cumsum(x))
  rsum <- (cx[(n+1):length(cx)] - cx[1:(length(cx) - n)]) / n

  return (rsum)
}

for(j in c(0.5,0.75,0.95)){
  q_table <- array(0,dim = c(H,W,4))
  reward <- NULL
  correction <- NULL

  for(i in 1:30000){
    foo <- q_learning(gamma = j, start_state = c(4,1))
    reward <- c(reward,foo[1])
    correction <- c(correction,foo[2])
  }

  vis_environment(i, gamma = j)
  plot(MovingAverage(reward,100),type = "l")
  plot(MovingAverage(correction,100),type = "l")
}
```

```
set.seed(12345)

for(j in c(0.5,0.75,0.95)){
  q_table <- array(0,dim = c(H,W,4))
  reward <- NULL
  correction <- NULL

  for(i in 1:30000){
    foo <- q_learning(epsilon = 0.1, gamma = j, start_state = c(4,1))
    reward <- c(reward,foo[1])
    correction <- c(correction,foo[2])
  }

  vis_environment(i, epsilon = 0.1, gamma = j)
  plot(MovingAverage(reward,100),type = "l")
  plot(MovingAverage(correction,100),type = "l")
}
```

A.4 Environment C: beta

```
# Environment C (the effect of beta).

set.seed(12345)

H <- 3
W <- 6

reward_map <- matrix(0, nrow = H, ncol = W)
reward_map[1,2:5] <- -1
reward_map[1,6] <- 10

q_table <- array(0,dim = c(H,W,4))

vis_environment()

for(j in c(0,0.2,0.4,0.66)){
  q_table <- array(0,dim = c(H,W,4))

  for(i in 1:10000)
    foo <- q_learning(gamma = 0.6, beta = j, start_state = c(1,1))

  vis_environment(i, gamma = 0.6, beta = j)
}
```

A.5 REINFORCE code source

The REINFORCE implementation, training runs and precomputed validation figures for Environments D and E are in `RL_Lab3_Colab.ipynb`.